UNIT I

Python Basics & Control Flow

History · Anaconda & Jupyter · Identifiers, variables, operators, data types · Input/Output · Type conversion · if / loops / try-except · Sample Programs 1–6

➤ What you will learn in this unit

- Where Python came from, what it is used for, and how to set up Anaconda and Jupyter Notebook.
- Building blocks: identifiers, keywords, variables, operators, data types, indentation, comments, input and output.
- Controlling a program with **if** statements, **while** / **for** loops, **break**, **continue** and **try–except**.
- Solutions of the sample programs **1 to 6** with real output.

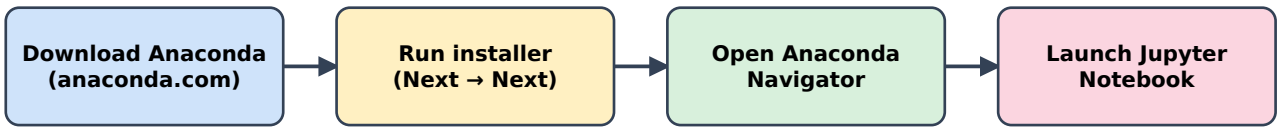
1. History and thrust areas of Python

- Created by **Guido van Rossum** at CWI, Netherlands; development began in **1989**, first release (0.9) in **1991**.
- Named after the comedy show *"Monty Python's Flying Circus"* — not after the snake.
- Python 2.0 (2000) → **Python 3.0 (2008)**. Python 2 is now retired; always use Python 3.
- Features: simple & readable, **interpreted**, free & open source, **dynamically typed**, object-oriented, portable, huge standard library.

Thrust (application) area	Examples / libraries
Data Science & Analytics	NumPy, Pandas, Matplotlib
Artificial Intelligence & Machine Learning	scikit-learn, TensorFlow, PyTorch
Web development	Django, Flask
Automation & scripting	Automating files, emails, testing
Desktop / games / IoT	Tkinter, Pygame, Raspberry Pi

2. Installing Anaconda and using Jupyter Notebook

Anaconda is a free Python distribution that installs Python, the **conda** package manager and popular data-science libraries (NumPy, Pandas, Matplotlib, Jupyter…) in one go.



Jupyter opens in your web browser — write code in cells, press Shift + Enter to run

- A Jupyter notebook (**.ipynb**) is made of **cells**: *Code* cells run Python, *Markdown* cells hold notes/headings.
- Output appears directly below the cell, so it is ideal for learning, data science and sharing results.

- Common shortcuts: **Shift+Enter** run cell · **A** / **B** insert cell above / below · **DD** delete cell · **M** Markdown · **Y** Code.

3. Parts of the Python language

3.1 Identifiers and keywords

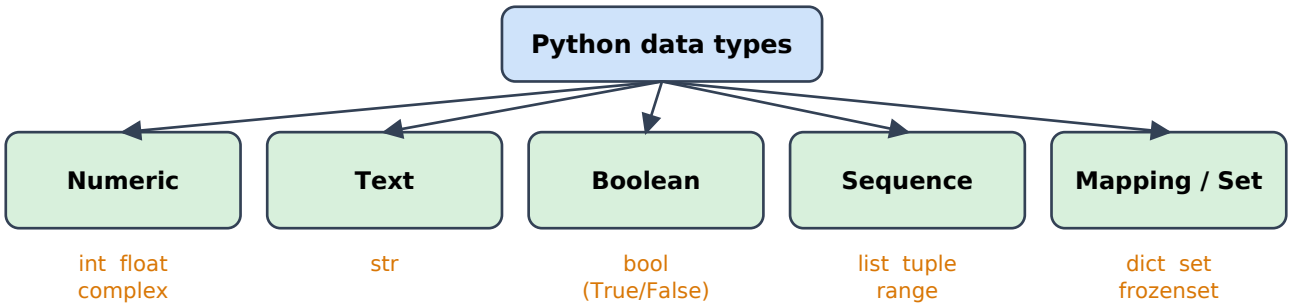
An **identifier** is the name given to a variable, function, class etc. Rules: it may contain letters, digits and underscore; it **cannot start with a digit**; it is **case-sensitive** (*Age* ≠ *age*); it cannot be a keyword and cannot contain spaces or symbols.

Keywords are reserved words with a special meaning, e.g. `if else elif while for break continue def class return True False None and or not in is try except`. Print them with `import keyword; print(keyword.kwlist)`.

3.2 Statements, expressions, variables

- **Expression** — produces a value: `3 + 4 * 2`. **Statement** — an instruction Python executes: `x = 10`, `print(x)`.
- **Variable** — a name that refers to a value in memory. No declaration needed: `x = 5` creates it; `x, y = 1, 2` assigns many.
- **Comments** — ignored by Python: `# single line` and `"""multi-line / docstring"""`.
- **Indentation** — Python uses **spaces (4 recommended)** to mark blocks. Wrong indentation gives *IndentationError*.

3.3 Data types



Type	Example	Mutable?	Note
int	`25, -3`	No	Whole numbers of any size
float	`3.14, 2e3`	No	Decimal numbers
complex	`2+3j`	No	Real + imaginary part (j)
str	`"Python"`	No	Sequence of characters
bool	`True, False`	No	Subclass of int (True = 1)
list	`[1, 2, 3]`	**Yes**	Ordered collection
tuple	`(1, 2, 3)`	No	Ordered, fixed
set	`{1, 2, 3}`	**Yes**	Unordered, unique items
dict	`{"a": 1}`	**Yes**	Key : value pairs

3.4 Operators, precedence and associativity

Type	Operators	Example → result
Arithmetic	`+ - * / // % **`	`7 // 2` → 3 , `7 % 2` → 1 , `2 ** 3` → 8
Relational	`== != > < >= <=`	`5 > 3` → True
Assignment	`= += -= *= /= //= %= **=`	`x += 2` means `x = x + 2`
Logical	`and or not`	`True and False` → False
Bitwise	`& | ^ ~ << >>`	`5 & 3` → 1 , `5 << 1` → 10
Ternary	`a if condition else b`	`"Pass" if m>=40 else "Fail"`
Membership	`in not in`	`"a" in "cat"` → True
Identity	`is is not`	Checks whether two names refer to the **same object**

Precedence (high → low): **() → ** → +x -x ~x → * / // % → + - → << >> → & → ^ → | → comparisons, in, is → not → and → or → assignment**.

- **Associativity** decides the order when operators have equal precedence. Most operators work **left to right**; the exponent **** and assignment work **right to left**.

```
print(2 + 3 * 4)          # * before +  -> 14
print((2 + 3) * 4)        # brackets first -> 20
print(2 ** 3 ** 2)        # right to left: 2 ** 9 -> 512
print(10 - 4 - 3)         # left to right: (10-4)-3 -> 3
print(not True or True)   # not first -> True
```

Output

```
14
20
512
3
True
```

3.5 Reading input and printing output

- **input(prompt)** always returns a **string**; convert it when you need numbers.
- **print()** shows output. Useful options: *sep* (separator), *end* (what to print at the end, default newline).

```
# name = input("Enter name: ")          # keyboard input (returns str)
# age = int(input("Enter age: "))       # convert to int
name, age = "Asha", 18                   # fixed values so this demo can run
print("Name:", name, "Age:", age)
print("a", "b", "c", sep="-")             # custom separator
print("Hello", end=" ")                   # no newline at end
print("World")
print(f"{name} will be {age + 1} next year")   # f-string
```

Output

```
Name: Asha Age: 18
a-b-c
Hello World
Asha will be 19 next year
```

3.6 Type conversion, type() and the is operator

Converting one data type to another is **type conversion** (casting). Functions: `int()` `float()` `str()` `bool()` `list()` `tuple()` `complex()`. Python also converts automatically (**implicit**) — e.g. int + float gives float.

```
a = "25"
b = int(a) + 5                # explicit conversion str -> int
print(b, type(b))
print(3 + 2.5, type(3 + 2.5))  # implicit conversion -> float
print(float(7), str(3.5), bool(0), bool("hi"), int(9.99))
x = [1, 2]; y = [1, 2]; z = x
print(x == y, x is y, x is z)  # == compares values, is compares identity
```

Output

```
30 <class 'int'>
5.5 <class 'float'>
7.0 3.5 False True 9
True False True
```

3.7 Dynamic and strongly typed language

- **Dynamically typed** — you do not declare a type; the type belongs to the **value** and is decided at run time, so the same variable can hold different types at different times.
- **Strongly typed** — Python does **not** silently mix incompatible types: `"5" + 2` raises *TypeError*; you must convert explicitly.

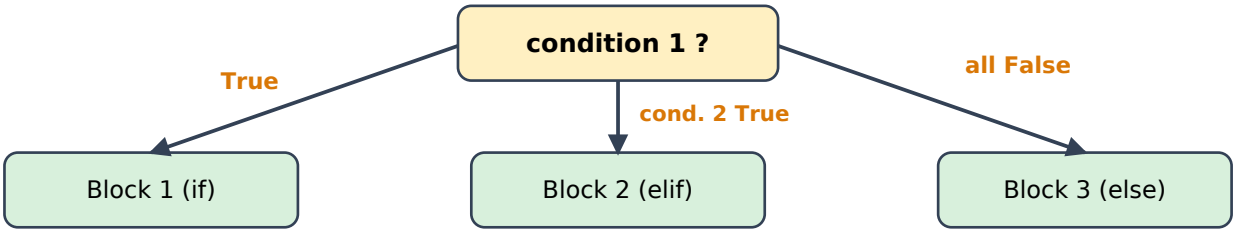
```
x = 10
print(type(x))
x = "ten"                     # same name, new type (dynamic typing)
print(type(x))
try:
    print("5" + 2)            # strong typing: no silent mixing
except TypeError as e:
    print("TypeError:", e)
```

Output

```
<class 'int'>
<class 'str'>
TypeError: can only concatenate str (not "int") to str
```

4. Control Flow Statements

4.1 Decision making: if, if-else, if…elif…else, nested if



Only ONE block runs. Python checks conditions from top to bottom.

```
marks = 72
if marks >= 90:
    print("Grade A")
```

```
elif marks >= 60:                 # checked only if the first is False
    print("Grade B")
else:
    print("Grade C")

n = 12
if n > 0:                          # nested if
    if n % 2 == 0:
        print("Positive even")
    else:
        print("Positive odd")
```

Output

```
Grade B
Positive even
```

4.2 Loops: while and for

	while loop	for loop
Use when	Number of repetitions is **not known**; repeat while condition is True	Iterating over a **sequence** / known range
Syntax	`while condition:`	`for item in sequence:`
Danger	Forgetting to update the variable → infinite loop	None — ends when the sequence ends

range(start, stop, step) gives numbers from *start* up to (but not including) *stop*: `range(5)` → 0..4, `range(1, 10, 3)` → 1, 4, 7.

```
i = 1
while i <= 3:                      # while loop
    print("while:", i)
    i += 1

for n in range(1, 10, 3):          # for loop with range
    print("for:", n)

for ch in "Hi":                    # looping over a string
    print(ch)
for i in range(2):                 # loop with else: runs when no break happens
    pass
else:
    print("loop finished normally")
```

Output

```
while: 1
while: 2
while: 3
for: 1
for: 4
for: 7
H
i
loop finished normally
```

4.3 break and continue

Statement	Effect
`break`	**Stops** the loop completely and jumps to the statement after the loop
`continue`	**Skips** the rest of the current iteration and goes to the next one
`pass`	Does nothing — a placeholder where a statement is required

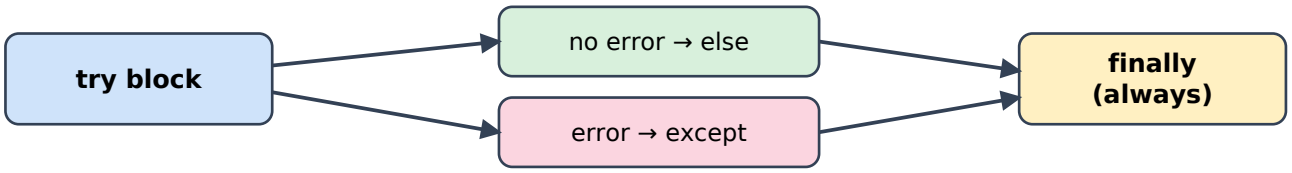
```
for i in range(1, 8):
    if i == 3:
        continue                    # skip 3
    if i == 6:
        break                       # stop at 6
    print(i, end=" ")
print()
```

Output

```
1 2 4 5
```

4.4 Catching exceptions with try and except

An **exception** is an error that occurs while the program runs (e.g. dividing by zero). Without handling, the program crashes. Put risky code in **try**; the **except** block runs only if an error occurs. **else** runs if no error; **finally** always runs.




```
try:
    a = int("abc")                  # raises ValueError
except ValueError:
    print("Please enter a valid number")
try:
    print(10 / 0)
except ZeroDivisionError as e:
    print("Error:", e)
else:
    print("no error")
finally:
    print("done")
```

Output

```
Please enter a valid number
Error: division by zero
done
```

★ Study Tip

- Remember: **Indentation = block**. A colon **:** must follow if / elif / else / while / for / def / try / except.
- For loops: **for** when you know how many times, **while** when you stop on a condition.

✓ Quick Revision

- Python: created by Guido van Rossum (1991), interpreted, dynamically & strongly typed, open source.
- Identifier rules: letters/digits/_ , cannot start with a digit, case-sensitive, not a keyword.
- Types: int, float, complex, str, bool, list, tuple, set, dict. **type()** shows the type; **is** compares identity, **==** compares value.
- Operators: arithmetic, relational, assignment, logical, bitwise, ternary, membership, identity. ** is right-associative.
- input() gives str → convert with int()/float(). print(sep=, end=).
- Control flow: if / elif / else, while, for with range(), break, continue, pass, try / except / else / finally.

SAMPLE PROGRAMS 1 – 6 (with output)

Program 1

Question

- Write a program to find the largest element among three numbers.

Program

```
# For keyboard input use: a = int(input("Enter first number: ")) etc.
a, b, c = 25, 78, 46

if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c

print("Numbers :", a, b, c)
print("Largest :", largest)
print("Using max():", max(a, b, c))
```

Output

```
Numbers : 25 78 46
Largest : 78
Using max(): 78
```

Program 2

Question

- Write a Program to display all prime numbers within an interval.

Program

```
start, end = 10, 50          # interval (can be read with input())

print("Prime numbers between", start, "and", end, "are:")
for num in range(start, end + 1):
    if num > 1:
        for i in range(2, int(num ** 0.5) + 1):   # check divisors up to sqrt(num)
            if num % i == 0:
                break                              # divisor found -> not prime
        else:
            print(num, end=" ")                    # loop ended without break -> prime
print()
```

Output

```
Prime numbers between 10 and 50 are:
11 13 17 19 23 29 31 37 41 43 47
```

i Remember

- A number is prime if it has no divisor between 2 and its square root. The **for…else** executes *else* only when the loop was not stopped by *break*.

Program 3

Question

- Write a program to swap two numbers without using a temporary variable.

Program

```
a, b = 10, 20
print("Before swap: a =", a, " b =", b)

# Method 1: tuple (multiple) assignment - the Pythonic way
a, b = b, a
print("After swap (tuple assignment): a =", a, " b =", b)

# Method 2: using arithmetic operators
a = a + b
b = a - b
a = a - b
print("After swap again (arithmetic): a =", a, " b =", b)

# Method 3: using XOR (bitwise)
a = a ^ b
b = a ^ b
a = a ^ b
print("After swap again (XOR)       : a =", a, " b =", b)
```

Output

```
Before swap: a = 10  b = 20
After swap (tuple assignment): a = 20  b = 10
After swap again (arithmetic): a = 10  b = 20
After swap again (XOR)       : a = 20  b = 10
```

Program 4

Question

- Demonstrate the following Operators in Python with suitable examples: i) Arithmetic ii) Relational iii) Assignment iv) Logical v) Bit wise vi) Ternary vii) Membership viii) Identity Operators.

Program

```
a, b = 17, 5

print("i) Arithmetic")
print("a + b =", a + b, "| a - b =", a - b, "| a * b =", a * b)
print("a / b =", a / b, "| a // b =", a // b, "| a % b =", a % b, "| a ** 2 =", a ** 2)

print("\nii) Relational")
print("a == b:", a == b, "| a != b:", a != b, "| a > b:", a > b, "| a <= b:", a <= b)

print("\niii) Assignment")
x = 10
x += 5;  print("x += 5  ->", x)
x -= 3;  print("x -= 3  ->", x)
x *= 2;  print("x *= 2  ->", x)
x //= 5; print("x //= 5 ->", x)
x **= 2; print("x **= 2 ->", x)

print("\niv) Logical")
print("a > 10 and b > 10:", a > 10 and b > 10)
print("a > 10 or  b > 10:", a > 10 or b > 10)
```

```
print("not (a > 10)     :", not (a > 10))

print("\nv) Bitwise  (a = 12 = 1100, b = 10 = 1010)")
p, q = 12, 10
print("p & q =", p & q, "| p | q =", p | q, "| p ^ q =", p ^ q)
print("~p =", ~p, "| p << 1 =", p << 1, "| p >> 1 =", p >> 1)

print("\nvi) Ternary")
marks = 65
print("Result:", "Pass" if marks >= 40 else "Fail")

print("\nvii) Membership")
fruits = ["apple", "mango"]
print("'apple' in fruits    :", "apple" in fruits)
print("'grape' not in fruits:", "grape" not in fruits)
print("'th' in 'python'     :", "th" in "python")

print("\nviii) Identity")
l1 = [1, 2]; l2 = [1, 2]; l3 = l1
print("l1 is l2:", l1 is l2, "| l1 is l3:", l1 is l3, "| l1 == l2:", l1 == l2)
print("l1 is not l2:", l1 is not l2)
```

Output

```
i) Arithmetic
a + b = 22 | a - b = 12 | a * b = 85
a / b = 3.4 | a // b = 3 | a % b = 2 | a ** 2 = 289

ii) Relational
a == b: False | a != b: True | a > b: True | a <= b: False

iii) Assignment
x += 5  -> 15
x -= 3  -> 12
x *= 2  -> 24
x //= 5 -> 4
x **= 2 -> 16

iv) Logical
a > 10 and b > 10: False
a > 10 or  b > 10: True
not (a > 10)     : False

v) Bitwise  (a = 12 = 1100, b = 10 = 1010)
p & q = 8 | p | q = 14 | p ^ q = 6
~p = -13 | p << 1 = 24 | p >> 1 = 6

vi) Ternary
Result: Pass

vii) Membership
'apple' in fruits    : True
'grape' not in fruits: True
'th' in 'python'     : True

viii) Identity
l1 is l2: False | l1 is l3: True | l1 == l2: True
l1 is not l2: True
```

Program 5

Question

- Write a program to add and multiply complex numbers.

Program

```
c1 = complex(3, 4)          # 3 + 4j
c2 = 2 + 5j                 # direct form

print("First number :", c1)
print("Second number:", c2)
print("Sum          :", c1 + c2)
print("Product      :", c1 * c2)
print("Real part of product:", (c1 * c2).real)
print("Imag part of product:", (c1 * c2).imag)
print("Magnitude of c1     :", abs(c1))
```

Output

```
First number : (3+4j)
Second number: (2+5j)
Sum          : (5+9j)
Product      : (-14+23j)
Real part of product: -14.0
Imag part of product: 23.0
Magnitude of c1     : 5.0
```

i Remember

- Product: (3+4j)(2+5j) = 6 + 15j + 8j + 20j² = 6 + 23j − 20 = **−14 + 23j** since j² = −1.

Program 6

Question

- Write a program to print multiplication table of a given number.

Program

```
n = 7                                  # or: n = int(input("Enter a number: "))
print("Multiplication table of", n)
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Output

```
Multiplication table of 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. Who developed Python? Mention any four features. 2. What are identifiers? State the naming rules. 3. Differentiate == and *is*. 4. What is dynamic typing? 5. Differentiate break and continue. 6. What is an exception? 7. Write the output of *print(2 ** 3 ** 2)*.	1. Explain the history and thrust areas of Python. 2. Explain the steps to install Anaconda and use Jupyter Notebook. 3. Explain the different data types with examples. 4. Explain all types of operators with precedence and associativity. 5. Explain if, if-else, elif and nested if with examples. 6. Explain while and for loops with break and continue. 7. Write programs 1–6 from the sample experiments.

UNIT II

Functions, Strings & Lists

Built-in functions & modules · Defining functions · Scope · Default, keyword, *args, **kwargs · Command-line arguments · Strings · Lists · Sample Programs 7–12

➤ What you will learn in this unit

- Writing and calling your own **functions**, passing arguments in different ways, and how variable **scope** works.
- Working with **strings** — indexing, slicing, methods and formatting.
- Working with **lists** — operations, slicing, built-in functions, methods and **del**.
- Solutions of the sample programs **7 to 12** with real output.

PART A — FUNCTIONS

1. Built-in functions and commonly used modules

A **function** is a named block of code that performs a task and can be reused. Python already provides many **built-in functions** — no import needed.

Function	Purpose	Example → result
`print() input()`	Output / input	
`len(x)`	Number of items	`len("abc")` → 3
`type(x) id(x)`	Type / memory identity	`type(5)` → int
`max() min() sum()`	Largest / smallest / total	`sum([1,2,3])` → 6
`abs() round() pow()`	Absolute value / rounding / power	`round(3.678, 2)` → 3.68
`sorted() reversed()`	Sort / reverse a sequence	`sorted([3,1,2])` → [1,2,3]
`range() enumerate() zip()`	Sequences and pairing	`list(range(3))` → [0,1,2]
`int() float() str() list()`	Type conversion	`int("7")` → 7

A **module** is a file of ready-made functions. Use **import** to bring it in: `import math`, `from math import sqrt`, `import random as r`.

Module	Commonly used functions
math	`sqrt() pow() ceil() floor() factorial() pi sin() log()`
random	`random() randint(a,b) choice(seq) shuffle(list)`
datetime	`datetime.now() date.today()`
sys	`sys.argv sys.exit() sys.version`
os	`os.getcwd() os.listdir() os.path`

```
import math, random
print(math.sqrt(49), math.ceil(4.2), math.floor(4.8), math.factorial(5))
```

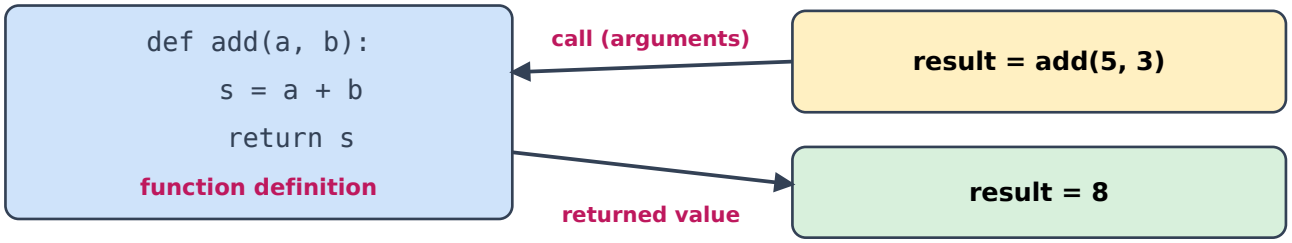
```
print(round(math.pi, 4))
random.seed(1)                          # fixed seed so the demo repeats
print(random.randint(1, 10) in range(1, 11))
print(abs(-7), pow(2, 5), max(4, 9, 2), sum([1, 2, 3]))
```

Output

```
7.0 5 4 120
3.1416
True
7 32 9 6
```

2. Function definition and calling; return statement; void function

Syntax: `def function_name(parameters):` followed by an **indented body**. The function runs only when it is **called**.



Term	Meaning
Parameter	Variable in the function definition (*a, b* above)
Argument	Actual value passed when calling (*5, 3*)
return statement	Sends a value back to the caller and **ends** the function
Void function	A function with no return statement (or plain *return*) — it gives back **None**; used for actions such as printing

```
def add(a, b):              # function with return value
    return a + b

def greet(name):            # void function (no return)
    print("Hello,", name)

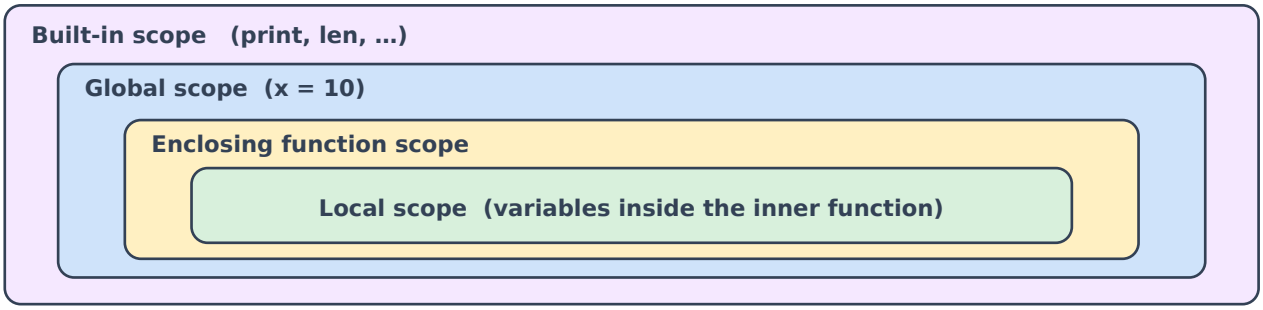
print(add(5, 3))
r = greet("Asha")
print(r)                    # void function returns None
```

Output

```
8
Hello, Asha
None
```

3. Scope and lifetime of variables

- **Scope** = the part of the program where a variable can be used. **Lifetime** = how long it stays in memory.
- **Local** variable — created inside a function; exists only **while the function runs**.
- **Global** variable — created outside all functions; lives until the program ends.
- To **change** a global variable inside a function, declare it with the **global** keyword.
- Python looks up names in the order **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in (LEGB rule).




```
x = 10                      # global

def show():
    y = 5                   # local
    print("inside :", x, y)

def change():
    global x                # modify the global variable
    x = 99

show()
change()
print("outside:", x)
# print(y)                  # NameError: y exists only inside show()
```

Output

```
inside : 10 5
outside: 99
```

4. Ways of passing arguments

Type	Description	Example call
Positional	Matched by **position** (order matters)	`power(2, 3)`
Default parameters	Parameter has a default value used when argument is missing	`def power(a, b=2)`
Keyword arguments	Pass by **name** — order does not matter	`power(b=3, a=2)`
***args**	Collects any number of extra positional arguments into a **tuple**	`def f(*args)`
****kwargs**	Collects extra keyword arguments into a **dictionary**	`def f(**kwargs)`

```
def power(a, b=2):                  # b has a default value
    return a ** b
print(power(5), power(2, 3), power(b=4, a=2))   # default / positional / keyword

def total(*args):                   # any number of positional arguments
    return sum(args)
```

```
print(total(1, 2, 3, 4))

def info(**kwargs):                    # any number of keyword arguments
    for k, v in kwargs.items():
        print(k, "=", v)
info(name="Asha", age=18)
```

Output

```
25 8 16
10
name = Asha
age = 18
```

! Careful!

- Parameters with default values must come **after** those without defaults: *def f(a, b=2)* is correct, *def f(a=2, b)* is an error.
- Order in a definition: normal → default → *args → **kwargs.

5. Command-line arguments

Values typed after the program name when running it from the terminal are **command-line arguments**. They are available in the list **sys.argv** (*argv[0]* is the script name; every value is a **string**).

```
# file: add.py
import sys
print("Script name:", sys.argv[0])
a = int(sys.argv[1])
b = int(sys.argv[2])
print("Sum =", a + b)
```

Run it as: `python add.py 10 20`

Output

```
Script name: add.py
Sum = 30
```

i Remember

- *len(sys.argv)* gives the number of arguments (including the script name).

PART B — STRINGS

6. Creating and storing strings

A **string** is a sequence of characters written in single, double or triple quotes. Strings are **immutable** — once created they cannot be changed (methods return a *new* string).

```
s1 = 'Python'
s2 = "Programming"
s3 = """This is a
multi-line string"""
print(s1, s2)
print(s3)
print("It's fine", 'He said "hi"', "Tab:\tNewline:\\n")   # quotes and escape sequences
```

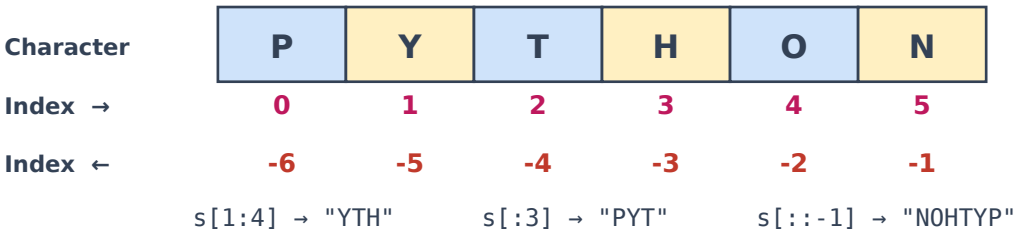
Output

```
Python Programming
This is a
multi-line string
It's fine He said "hi" Tab:	Newline:\n
```

Basic string operations

Operation	Example	Result
Concatenation +	`"Hello" + " World"`	Hello World
Repetition *	`"ab" * 3`	ababab
Length len()	`len("Python")`	6
Membership in	`"th" in "Python"`	True
Comparison	`"apple" < "banana"`	True (dictionary order)

7. Accessing characters, slicing and joining



- **Indexing**: *s[0]* is the first character, *s[-1]* the last. Index out of range → *IndexError*.
- **Slicing**: *s[start:stop:step]* — start included, **stop excluded**. Missing start = 0, missing stop = end, *step* = *-1* reverses.
- **Joining**: "*sep*".*join(list_of_strings)* combines items using the separator; *split()* does the opposite.

```
s = "PYTHON"
print(s[0], s[-1], s[2])
print(s[1:4], s[:3], s[3:], s[::2], s[::-1])
words = ["Learn", "Python", "Fast"]
joined = " ".join(words)
print(joined)
print(joined.split(" "))
```

Output

```
P N T
YTH PYT HON PTO NOHTYP
Learn Python Fast
['Learn', 'Python', 'Fast']
```

8. String methods and formatting

Method	Purpose	Example → result
`upper() lower() title()`	Change case	`"hi".upper()` → HI
`capitalize() swapcase()`	First capital / swap case	`"python".capitalize()` → Python
`strip() lstrip() rstrip()`	Remove spaces at ends	`" a ".strip()` → a
`find(sub) index(sub)`	Position of substring (find gives -1 if absent)	`"hello".find("l")` → 2
`count(sub)`	Number of occurrences	`"banana".count("a")` → 3
`replace(old, new)`	Replace text	`"cat".replace("c","b")` → bat
`startswith() endswith()`	Check beginning / end	`"abc".startswith("a")` → True
`isdigit() isalpha() isalnum()`	Check kind of characters	`"123".isdigit()` → True
`split() join()`	String ↔ list	`"a b".split()` → ['a','b']

Formatting strings

```
name, marks = "Asha", 92.456
print("Name: %s, Marks: %.1f" % (name, marks))        # % formatting
print("Name: {}, Marks: {:.2f}".format(name, marks))  # str.format()
print(f"Name: {name}, Marks: {marks:.2f}")            # f-string (best)
print(f"{name:>8}|{name:<8}|{name:^8}|")              # alignment
print(f"{255:b} {255:x} {1234567:,}")                 # binary, hex, thousands
```

Output

```
Name: Asha, Marks: 92.5
Name: Asha, Marks: 92.46
Name: Asha, Marks: 92.46
    Asha|Asha    |  Asha  |
11111111 ff 1,234,567
```

PART C — LISTS

9. Lists: creating and basic operations

A **list** is an ordered, **mutable** collection written in square brackets. It can hold items of different types and allows duplicates.

```
nums = [10, 20, 30, 40, 50]
mixed = [1, "two", 3.0, [4, 5]]          # lists can hold any type, even lists
print(nums, len(mixed))
print(nums + [60], [0] * 3)              # concatenation and repetition
print(20 in nums, 99 in nums)            # membership
nums[1] = 25                             # lists are mutable
print(nums)
for item in nums:                        # iteration
```

```
    print(item, end=" ")
print()
```

Output

```
[10, 20, 30, 40, 50] 4
[10, 20, 30, 40, 50, 60] [0, 0, 0]
True False
[10, 25, 30, 40, 50]
10 25 30 40 50
```

Indexing and slicing in lists

Works exactly like strings: *L[i]*, negative indexes, and *L[start:stop:step]*. A slice returns a **new list**.

```
L = [10, 20, 30, 40, 50, 60]
print(L[0], L[-1], L[1:4], L[:2], L[3:], L[::2], L[::-1])
L[1:3] = [21, 31]                          # slice assignment
print(L)
```

Output

```
10 60 [20, 30, 40] [10, 20] [40, 50, 60] [10, 30, 50] [60, 50, 40, 30, 20, 10]
[10, 21, 31, 40, 50, 60]
```

Built-in functions used on lists

Function	Result
`len(L) max(L) min(L) sum(L)`	Size, largest, smallest, total of items
`sorted(L)`	A **new** sorted list (original unchanged)
`list(x)`	Converts a string / tuple / range into a list
`any(L) all(L)`	True if any / all items are True
`enumerate(L) reversed(L)`	Index–value pairs / reverse iterator

List methods

Method	Purpose
`append(x)`	Add x at the **end**
`insert(i, x)`	Insert x at index i
`extend(iterable)`	Add all items of another list
`remove(x)`	Delete the **first** occurrence of value x
`pop(i)`	Remove and **return** item at index i (last if i omitted)
`index(x) count(x)`	Position of x / number of occurrences
`sort() reverse()`	Sort / reverse the list **in place**
`copy() clear()`	Shallow copy / remove everything

```
L = [30, 10, 20]
L.append(40); L.insert(1, 15); L.extend([5, 5])
print(L)
L.remove(5); print(L.pop(), L)
L.sort(); print(L)
L.reverse(); print(L, L.index(20), L.count(10))
```

Output

```
[30, 15, 10, 20, 40, 5, 5]
5 [30, 15, 10, 20, 40]
[10, 15, 20, 30, 40]
[40, 30, 20, 15, 10] 2 1
```

The del statement

del deletes an item, a slice, or the whole variable. (*remove()* deletes by value, *pop()* by index and returns it, *del* by index/slice and returns nothing.)

```
L = [10, 20, 30, 40, 50, 60]
del L[0]                  # delete by index
print(L)
del L[1:3]                # delete a slice
print(L)
del L                     # delete the whole list
# print(L)               # NameError: name 'L' is not defined
```

Output

```
[20, 30, 40, 50, 60]
[20, 50, 60]
```

★ Study Tip

- Strings are **immutable**, lists are **mutable** — a favourite exam question.
- **sort()** changes the list and returns None; **sorted()** returns a new list.

✓ Quick Revision

- **def name(params):** … **return**. No return → None (void function). Local vs global scope (global keyword), LEGB.
- Arguments: positional, default, keyword, ***args** (tuple), ****kwargs** (dict). Command-line arguments → **sys.argv**.
- Strings: immutable; index s[i], slice s[a:b:c]; methods upper, lower, strip, find, count, replace, split, join; f-strings for formatting.
- Lists: mutable; append, insert, extend, remove, pop, sort, reverse, index, count; del removes by index/slice.
- Built-ins on lists: len, max, min, sum, sorted.

SAMPLE PROGRAMS 7 – 12 (with output)

Program 7

Question

- Write a program to define a function with multiple return values.

Program

```
def calculate(a, b):
    add = a + b
    sub = a - b
    mul = a * b
    div = a / b
    return add, sub, mul, div        # returns a tuple of four values

result = calculate(20, 4)
print("Returned tuple:", result)

s, d, m, q = calculate(20, 4)        # unpack the values
print("Sum        =", s)
print("Difference =", d)
print("Product    =", m)
print("Quotient   =", q)
```

Output

```
Returned tuple: (24, 16, 80, 5.0)
Sum        = 24
Difference = 16
Product    = 80
Quotient   = 5.0
```

i Remember

- Python returns multiple values as a **tuple**; they can be unpacked into separate variables.

Program 8

Question

- Write a program to define a function using default arguments.

Program

```
def student_info(name, course="B.Tech", year=1):     # default arguments
    print(f"Name: {name}, Course: {course}, Year: {year}")

student_info("Asha")                          # uses both defaults
student_info("Ravi", "AI & ML")               # overrides course
student_info("Kiran", year=2)                 # keyword argument
student_info(year=3, name="Divya", course="CSE")

def interest(principal, rate=5, time=2):
    return principal * rate * time / 100
print("Interest with defaults:", interest(10000))
print("Interest with rate=8  :", interest(10000, 8))
```

Output

```
Name: Asha, Course: B.Tech, Year: 1
Name: Ravi, Course: AI & ML, Year: 1
Name: Kiran, Course: B.Tech, Year: 2
Name: Divya, Course: CSE, Year: 3
Interest with defaults: 1000.0
Interest with rate=8  : 1600.0
```

Program 9

Question

- Write a program to find the length of the string without using any library functions.

Program

```
s = "Python Programming"

count = 0
for ch in s:            # visit every character and count it
    count += 1

print("String :", s)
print("Length :", count)
```

Output

```
String : Python Programming
Length : 18
```

i Remember

- Here *len()* is not used — each character is counted with a loop.

Program 10

Question

- Write a program to check if the substring is present in a given string or not.

Program

```
text = "Python is easy to learn"
sub = "easy"

# Method 1: membership operator
if sub in text:
    print(f"'{sub}' is present in the string")
else:
    print(f"'{sub}' is not present")

# Method 2: find() returns the position or -1
pos = text.find("hard")
print("find('hard') =", pos, "-> not present" if pos == -1 else "-> present")

# Method 3: manual check without library methods
sub2 = "learn"
found = False
for i in range(len(text) - len(sub2) + 1):
    if text[i:i + len(sub2)] == sub2:
        found = True
        break
```

```
print(f"'{sub2}' present (manual check):", found)
```

Output

```
'easy' is present in the string
find('hard') = -1 -> not present
'learn' present (manual check): True
```

Program 11

Question

- Write a program to perform the given operations on a list: i. Addition ii. Insertion iii. slicing

Program

```
L = [10, 20, 30, 40, 50]
print("Original list:", L)

# i. Addition (adding elements)
L.append(60)                    # add one item at the end
L.extend([70, 80])              # add many items
print("After addition  :", L)
print("List + list     :", L + [90, 100])

# ii. Insertion (at a given position)
L.insert(2, 25)                 # insert 25 at index 2
print("After insertion :", L)

# iii. Slicing
print("L[2:5]  :", L[2:5])
print("L[:3]   :", L[:3])
print("L[4:]   :", L[4:])
print("L[::2]  :", L[::2])
print("L[::-1] :", L[::-1])
```

Output

```
Original list: [10, 20, 30, 40, 50]
After addition  : [10, 20, 30, 40, 50, 60, 70, 80]
List + list     : [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
After insertion : [10, 20, 25, 30, 40, 50, 60, 70, 80]
L[2:5]  : [25, 30, 40]
L[:3]   : [10, 20, 25]
L[4:]   : [40, 50, 60, 70, 80]
L[::2]  : [10, 25, 40, 60, 80]
L[::-1] : [80, 70, 60, 50, 40, 30, 25, 20, 10]
```

Program 12

Question

- Write a program to perform any 5 built-in functions by taking any list.

Program

```
nums = [45, 12, 78, 33, 5, 91]

print("List        :", nums)
print("len()       :", len(nums))              # number of items
print("max()       :", max(nums))              # largest
print("min()       :", min(nums))              # smallest
print("sum()       :", sum(nums))              # total
print("sorted()    :", sorted(nums))           # ascending (new list)
print("sorted(rev) :", sorted(nums, reverse=True))
print("Average     :", round(sum(nums) / len(nums), 2))
```

Output

```
List        : [45, 12, 78, 33, 5, 91]
len()       : 6
max()       : 91
min()       : 5
sum()       : 264
sorted()    : [5, 12, 33, 45, 78, 91]
sorted(rev) : [91, 78, 45, 33, 12, 5]
Average     : 44.0
```

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. What is a function? Give its syntax. 2. Differentiate parameter and argument. 3. What is a void function? 4. What are *args and **kwargs? 5. Differentiate local and global variables. 6. What is string slicing? Give an example. 7. Differentiate sort() and sorted(). 8. Differentiate remove(), pop() and del.	1. Explain the types of function arguments with examples. 2. Explain scope and lifetime of variables with the LEGB rule. 3. Explain command-line arguments with a program. 4. Explain any ten string methods with examples. 5. Explain the different ways of formatting strings. 6. Explain list methods and built-in functions with examples. 7. Write programs 7–12 from the sample experiments.

UNIT III

Dictionaries, Tuples & Sets

Dictionary operations & methods · Tuples · Tuple–list–dictionary relations · zip() · Sets · Set methods · Frozenset · Sample Programs 13–17

➤ What you will learn in this unit

- Storing data as **key : value** pairs in **dictionaries** and using their functions, methods and **del**.
- Creating **tuples**, their operations, and how they relate to lists and dictionaries; the **zip()** function.
- Working with **sets** (unique items, mathematical operations) and **frozenset**.
- Solutions of the sample programs **13 to 17** with real output.

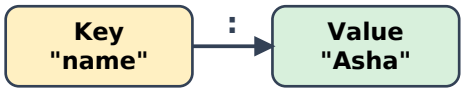
PART A — DICTIONARIES

1. Creating a dictionary

A **dictionary** stores data as **key : value** pairs inside curly braces. Keys must be **unique and immutable** (str, int, tuple); values can be anything. Since Python 3.7 a dictionary remembers the **insertion order**.

student (dictionary)

```
"name"   :  "Asha"
 "age"   :  18
"city"   :  "Kadapa"
```



one key-value pair

Access a value with its KEY: student["name"]

```
student = {"name": "Asha", "age": 18, "city": "Kadapa"}   # using { }
empty = {}                                                # empty dictionary
d2 = dict(a=1, b=2)                                       # dict() function
d3 = dict([("x", 10), ("y", 20)])                         # from list of pairs
print(student)
print(d2, d3, type(empty))
```

Output

```
{'name': 'Asha', 'age': 18, 'city': 'Kadapa'}
{'a': 1, 'b': 2} {'x': 10, 'y': 20} <class 'dict'>
```

2. Accessing and modifying key : value pairs

Task	Code	Note
Access value	`d["name"]`	KeyError if key is missing
Safe access	`d.get("phone", "NA")`	Returns default (None) if missing — no error
Modify value	`d["age"] = 19`	Key exists → value is updated
Add new pair	`d["email"] = "a@x.com"`	Key does not exist → new pair added
Check key	`"age" in d`	Membership tests the **keys**
Loop	`for k in d: / d.items()`	Loop over keys / (key, value) pairs

```
d = {"name": "Asha", "age": 18}
print(d["name"], d.get("phone", "NA"))
d["age"] = 19                       # modify
d["city"] = "Kadapa"                # add
print(d)
for k, v in d.items():              # iterate over pairs
    print(k, "->", v)
```

Output

```
Asha NA
{'name': 'Asha', 'age': 19, 'city': 'Kadapa'}
name -> Asha
age -> 19
city -> Kadapa
```

3. Built-in functions and dictionary methods

Built-in function	Result
`len(d)`	Number of key-value pairs
`sorted(d)`	List of keys in sorted order
`max(d) min(d)`	Largest / smallest **key**
`sum(d)`	Sum of the keys (if numeric)
`list(d) tuple(d)`	Keys as a list / tuple
`any(d) all(d)`	Truth test on keys

Method	Purpose
`keys() values() items()`	View of all keys / all values / all (key, value) pairs
`get(key, default)`	Value for key or default if absent
`update(other)`	Add / overwrite pairs from another dictionary
`pop(key)`	Remove key and **return** its value
`popitem()`	Remove and return the **last inserted** pair
`setdefault(key, v)`	Return value if key exists, otherwise add key with v
`copy() clear()`	Shallow copy / remove all items
`dict.fromkeys(keys, v)`	New dictionary with the given keys, all with value v

```
d = {"a": 1, "b": 2, "c": 3}
print(len(d), sorted(d), max(d), sum(d.values()))
```

```
print(list(d.keys()), list(d.values()), list(d.items()))
d.update({"d": 4, "a": 10})
print(d)
print(d.pop("b"), d.popitem(), d)
print(dict.fromkeys(["x", "y"], 0))
```

Output

```
3 ['a', 'b', 'c'] c 6
['a', 'b', 'c'] [1, 2, 3] [('a', 1), ('b', 2), ('c', 3)]
{'a': 10, 'b': 2, 'c': 3, 'd': 4}
2 ('d', 4) {'a': 10, 'c': 3}
{'x': 0, 'y': 0}
```

The del statement

```
d = {"name": "Asha", "age": 18, "city": "Kadapa"}
del d["city"]                # delete one pair by key
print(d)
d.clear()                    # remove all pairs
print(d)
del d                        # delete the dictionary variable itself
```

Output

```
{'name': 'Asha', 'age': 18}
{}
```

i Remember

- **pop()** returns the removed value, **del** does not. Deleting a missing key gives *KeyError*.

PART B — TUPLES

4. Creating tuples and basic operations

A **tuple** is an ordered collection like a list, but it is **immutable** — items cannot be changed, added or removed after creation. It is written in round brackets. Tuples are faster than lists and safe for fixed data.

```
t1 = (10, 20, 30)
t2 = 5, 6, 7                    # brackets are optional
single = (99,)                  # one-item tuple NEEDS a comma
t3 = tuple([1, 2, 3])           # tuple() function: list -> tuple
t4 = tuple("abc")               # string -> tuple
print(t1, t2, single, t3, t4)
print(type((99)), type((99,)))
```

Output

```
(10, 20, 30) (5, 6, 7) (99,) (1, 2, 3) ('a', 'b', 'c')
<class 'int'> <class 'tuple'>
```

Operation	Example	Result
Concatenation	`(1,2) + (3,4)`	(1, 2, 3, 4)
Repetition	`(1,2) * 2`	(1, 2, 1, 2)
Membership	`3 in (1,2,3)`	True
Length	`len((1,2,3))`	3
Unpacking	`a, b = (5, 6)`	a = 5, b = 6
Change an item	`t[0] = 5`	**TypeError** (immutable)

5. Indexing, slicing and built-in functions on tuples

Indexing and slicing are the same as for lists and strings: *t[0]*, *t[-1]*, *t[1:3]*, *t[::-1]*.

Function / method	Result
`len(t) max(t) min(t) sum(t)`	Size, largest, smallest, total
`sorted(t)`	Sorted **list** of the items (tuple itself is not changed)
`t.count(x)`	Number of times x occurs
`t.index(x)`	Index of the first occurrence of x
`tuple(seq)`	Convert a list / string / range into a tuple

```
t = (4, 8, 15, 8, 23, 42)
print(t[0], t[-1], t[1:4], t[::-1])
print(len(t), max(t), min(t), sum(t))
print(t.count(8), t.index(15), sorted(t))
```

Output

```
4 42 (8, 15, 8) (42, 23, 8, 15, 8, 4)
6 42 4 100
2 2 [4, 8, 8, 15, 23, 42]
```

6. Relation between tuples and lists

	List	Tuple
Brackets	`[ ]`	`( )`
Mutable?	**Yes** — can add, remove, change	**No** — fixed once created
Methods	Many (append, remove, sort …)	Only count() and index()
Speed / memory	Slower, more memory	Faster, less memory
Use for	Data that changes	Fixed data (coordinates, dates), dictionary keys

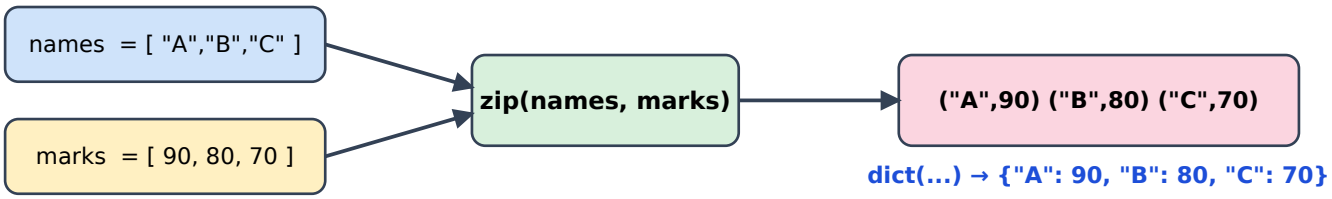
```
L = [1, 2, 3]
T = tuple(L)              # list -> tuple
print(T)
L2 = list(T)              # tuple -> list
L2.append(4)              # now it can change
print(L2)
a, b = 5, 10
a, b = b, a               # swapping uses tuple packing / unpacking
print(a, b)
```

Output

```
(1, 2, 3)
[1, 2, 3, 4]
10 5
```

7. Relation between tuples and dictionaries; the zip() function

- **Tuples as keys**: a tuple is immutable (hashable) so it can be a dictionary key; a list cannot.
- **items()** returns the dictionary as a sequence of **(key, value) tuples**; `dict()` converts a sequence of tuples back into a dictionary.
- **zip(a, b, …)** pairs up items of several sequences into **tuples** — handy to build a dictionary from two lists.




```
names = ["Asha", "Ravi", "Kiran"]
marks = [90, 80, 70]
pairs = list(zip(names, marks))             # list of tuples
print(pairs)
d = dict(zip(names, marks))                 # two lists -> dictionary
print(d)
print(list(d.items()))                      # dictionary -> list of tuples
loc = {(17.38, 78.48): "Hyderabad"}         # tuple as a key
print(loc[(17.38, 78.48)])
for n, m in zip(names, marks):              # loop over two lists together
    print(n, m)
```

Output

```
[('Asha', 90), ('Ravi', 80), ('Kiran', 70)]
{'Asha': 90, 'Ravi': 80, 'Kiran': 70}
[('Asha', 90), ('Ravi', 80), ('Kiran', 70)]
Hyderabad
Asha 90
Ravi 80
Kiran 70
```

PART C — SETS

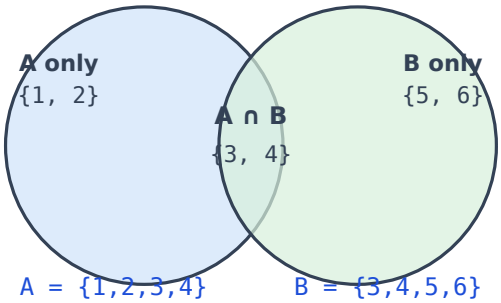
8. Creating sets and set methods

A **set** is an **unordered** collection of **unique** items written in curly braces. No indexing, no duplicates; the set itself is mutable but its items must be immutable.

```
s = {5, 1, 3, 3, 1}                 # duplicates are removed automatically
print(s, len(s))
e = set()                           # empty set  ({} would create a dictionary!)
t = set([1, 2, 2, 3])               # set() from a list - quick way to remove duplicates
print(e, t, type({}))
print(3 in s)
```

Output

```
{1, 3, 5} 3
set() {1, 2, 3} <class 'dict'>
True
```




```
A | B   union          → {1,2,3,4,5,6}
A & B   intersection   → {3,4}
A - B   difference     → {1,2}
A ^ B   symmetric diff → {1,2,5,6}
```

Method	Operator	Purpose
`add(x)`		Add one item
`update(iterable)`		Add many items
`remove(x)`		Delete x — **KeyError** if absent
`discard(x)`		Delete x — no error if absent
`pop()` `clear()`		Remove an arbitrary item / everything
`union(B)`	`A | B`	All items of both sets
`intersection(B)`	`A & B`	Items common to both
`difference(B)`	`A - B`	Items in A but not in B
`symmetric_difference(B)`	`A ^ B`	Items in either set but not in both
`issubset(B)` `issuperset(B)` `isdisjoint(B)`	`A <= B`	Subset / superset / no common items

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B, A & B, A - B, A ^ B)
print(A.union(B) == A | B)
A.add(10); A.discard(1); A.remove(2)
print(A)
print({3, 4}.issubset(B), A.isdisjoint({100}))
```

Output

```
{1, 2, 3, 4, 5, 6} {3, 4} {1, 2} {1, 2, 5, 6}
True
{3, 4, 10}
True True
```

9. Frozenset

A **frozenset** is an **immutable** set. It supports all read-only set operations (union, intersection ...) but not add / remove. Because it is hashable it can be used as a **dictionary key** or as an item of another set.

```
f = frozenset([1, 2, 3, 2])
print(f, type(f))
print(f | {4}, 2 in f)
try:
    f.add(5)
except AttributeError as e:
    print("Error:", e)
d = {frozenset([1, 2]): "pair"}       # frozenset used as dictionary key
print(d[frozenset([2, 1])])
```

Output

```
frozenset({1, 2, 3}) <class 'frozenset'>
frozenset({1, 2, 3, 4}) True
Error: 'frozenset' object has no attribute 'add'
pair
```

	List	Tuple	Set	Dictionary
Brackets	`[ ]`	`( )`	`{ }`	`{k: v}`
Ordered	Yes	Yes	No	Yes (insertion order)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	By position	By position	No	By key

★ Study Tip

- An **empty {}** is a dictionary — use **set()** for an empty set, and **(x,)** with a comma for a one-item tuple.
- Set operators: **|** union, **&** intersection, **-** difference, **^** symmetric difference.

✓ Quick Revision

- **Dictionary**: {key: value}; keys unique & immutable; get(), keys(), values(), items(), update(), pop(), popitem(); del d[key].
- **Tuple**: immutable, ( ); count() and index() only; one-item tuple needs a comma; tuple ↔ list conversion.
- **zip()** pairs items → dict(zip(keys, values)). **items()** gives (key, value) tuples.
- **Set**: unique, unordered, no indexing; | & - ^; add / remove / discard; **frozenset** = immutable set.

SAMPLE PROGRAMS 13 – 17 (with output)

Program 13

Question

- Write a program to create tuples (name, age, address, college) for at least two members and concatenate the tuples.

Program

```
member1 = ("Asha", 18, "Kadapa", "JNTUA College")      # (name, age, address, college)
member2 = ("Ravi", 19, "Tirupati", "JNTUA College")
member3 = ("Kiran", 18, "Anantapur", "JNTUA College")

print("Member 1:", member1)
print("Member 2:", member2)

combined = member1 + member2                           # concatenation of tuples
print("\nConcatenated tuple (2 members):\n", combined)

all_members = member1 + member2 + member3
print("\nConcatenated tuple (3 members):\n", all_members)
print("Total items:", len(all_members))
```

Output

```
Member 1: ('Asha', 18, 'Kadapa', 'JNTUA College')
Member 2: ('Ravi', 19, 'Tirupati', 'JNTUA College')

Concatenated tuple (2 members):
 ('Asha', 18, 'Kadapa', 'JNTUA College', 'Ravi', 19, 'Tirupati', 'JNTUA College')

Concatenated tuple (3 members):
 ('Asha', 18, 'Kadapa', 'JNTUA College', 'Ravi', 19, 'Tirupati', 'JNTUA College', 'Kiran', 18, 'Anantapur', 'JN
Total items: 12
```

i Remember

- Tuples are immutable, so **+** does not change the old tuples — it creates a **new** tuple.

Program 14

Question

- Write a program to count the number of vowels in a string (No control flow allowed).

Program

```
s = "Python Programming Is Fun"
text = s.lower()

# No if / for / while: use count() for each vowel and add the results
vowels = text.count("a") + text.count("e") + text.count("i") + text.count("o") + text.count("u")
print("String :", s)
print("Vowels :", vowels)

# Another way without any control flow statement
print("Vowels (map + sum):", sum(map(text.count, "aeiou")))
```

Output

```
String : Python Programming Is Fun
Vowels : 6
Vowels (map + sum): 6
```

i Remember

- **Logic:** *str.count()* counts how many times a character appears, so no loop or *if* is needed. *map()* applies count to each vowel.

Program 15

Question

- Write a program to check if a given key exists in a dictionary or not.

Program

```
student = {"name": "Asha", "age": 18, "city": "Kadapa"}

key = "age"
if key in student:                      # membership operator checks keys
    print(f"Key '{key}' exists, its value is {student[key]}")
else:
    print(f"Key '{key}' does not exist")

key = "phone"
print(f"Key '{key}' exists:", key in student)
print("Using get():", student.get("phone", "Key not found"))
```

Output

```
Key 'age' exists, its value is 18
Key 'phone' exists: False
Using get(): Key not found
```

Program 16

Question

- Write a program to add a new key-value pair to an existing dictionary.

Program

```
student = {"name": "Asha", "age": 18}
print("Original  :", student)

student["city"] = "Kadapa"                  # method 1: assignment
print("After add :", student)

student.update({"branch": "AI & ML"})       # method 2: update()
print("After update():", student)

student.setdefault("year", 2)               # method 3: setdefault()
print("After setdefault():", student)
```

Output

```
Original  : {'name': 'Asha', 'age': 18}
After add : {'name': 'Asha', 'age': 18, 'city': 'Kadapa'}
After update(): {'name': 'Asha', 'age': 18, 'city': 'Kadapa', 'branch': 'AI & ML'}
After setdefault(): {'name': 'Asha', 'age': 18, 'city': 'Kadapa', 'branch': 'AI & ML', 'year': 2}
```

Program 17

Question

- Write a program to sum all the items in a given dictionary.

Program

```
sales = {"Jan": 1200, "Feb": 1500, "Mar": 900, "Apr": 1800}

total = 0
for month in sales:                          # loop method
    total += sales[month]
print("Sum using loop    :", total)

print("Sum using sum()   :", sum(sales.values()))
print("Average           :", sum(sales.values()) / len(sales))
```

Output

```
Sum using loop    : 5400
Sum using sum()   : 5400
Average           : 1350.0
```

i Remember

- Use **d.values()** for the values. *sum(d)* would add the **keys**, not the values.

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. What is a dictionary? Can a list be a key? 2. Differentiate pop() and del for dictionaries. 3. What is the use of get()? 4. Differentiate list and tuple. 5. How do you create a one-item tuple? 6. What is zip()? 7. Differentiate remove() and discard(). 8. What is a frozenset?	1. Explain dictionary creation, access and modification with examples. 2. Explain any eight dictionary methods with examples. 3. Explain tuples with operations and built-in functions. 4. Explain the relation between tuples, lists and dictionaries. 5. Explain set methods and mathematical set operations with examples. 6. Compare list, tuple, set and dictionary. 7. Write programs 13–17 from the sample experiments.

UNIT IV

Files & Object-Oriented Programming

Text & binary files · pickle · CSV · os and os.path · Classes, objects, constructors · Encapsulation · Inheritance · Polymorphism · Sample Programs 18–23

➤ What you will learn in this unit

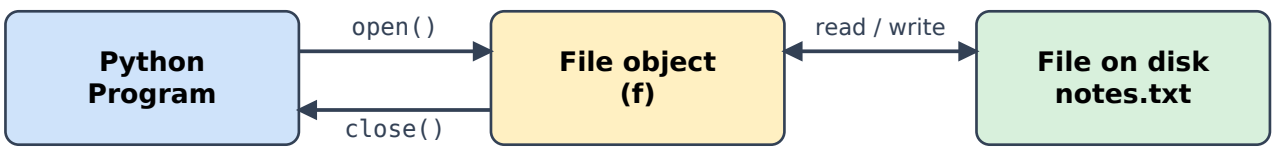
- How Python stores data permanently — reading and writing text, binary and CSV files.
- How to save whole Python objects with the **pickle** module and work with folders using **os** / **os.path**.
- The OOP way of thinking: classes, objects, constructors, attributes, encapsulation, inheritance and polymorphism.
- Solutions of the sample programs **18 to 23** with real output.

PART A — FILES

1. Why do we need files? Types of files

Variables live in RAM, so everything is lost when the program ends. A **file** is a named place on the disk where data is stored **permanently**. Python treats every file as a stream of data that can be read or written.

	Text file	Binary file
Content	Human-readable characters (letters, digits, symbols)	Raw bytes (0s and 1s) — not readable in a text editor
Examples	.txt, .py, .csv, .html	.jpg, .png, .mp3, .pdf, .exe, .pkl
Line ending	Each line ends with newline character (**\n**)	No concept of lines
Open modes	`"r" "w" "a"`	`"rb" "wb" "ab"`
Handled with	read(), readline(), write()	read(), write() with **bytes**; pickle module



Every file operation: OPEN → PROCESS (read / write) → CLOSE

2. Opening and closing a file

Syntax: `file_object = open(filename, mode)`. The **mode** tells Python what you want to do with the file.

Mode	Meaning	If file does not exist	Old content
`r`	Read only (default)	Error (FileNotFoundError)	Kept
`w`	Write only	Creates a new file	**Erased** (overwritten)
`a`	Append — add at the end	Creates a new file	Kept
`r+`	Read and write	Error	Kept
`w+`	Write and read	Creates a new file	**Erased**
`a+`	Append and read	Creates a new file	Kept
`rb wb ab`	Same as r / w / a but in **binary** mode	As above	As above

Always close a file after use so that data is saved properly. The safest way is the **with** statement — it closes the file **automatically**, even if an error occurs.

```
f = open("notes.txt", "w")      # open
f.write("Hello Python")         # process
f.close()                       # close

with open("notes.txt", "r") as f:   # recommended: closes automatically
    print(f.read())
    print(f.name, f.mode, f.closed) # file attributes
```

Output

```
Hello Python
notes.txt r False
```

3. Reading and writing text data

Reading methods

Method	What it does
`f.read()`	Reads the **entire** file as one string. *read(n)* reads only n characters.
`f.readline()`	Reads **one line** (including the newline) each time it is called.
`f.readlines()`	Reads all lines and returns them as a **list** of strings.
`for line in f:`	Reads the file line by line — best for big files (memory-friendly).

Writing methods

Method	What it does
`f.write(s)`	Writes the string *s*. Does **not** add a newline automatically. Returns number of characters written.
`f.writelines(list)`	Writes every string of a list (add "\n" yourself).
`f.tell()`	Returns the current position of the file pointer.
`f.seek(n)`	Moves the file pointer to byte number n (e.g. seek(0) goes back to start).

```
with open("data.txt", "w") as f:
    f.write("Line 1\n")
    f.writelines(["Line 2\n", "Line 3\n"])

with open("data.txt", "a") as f:        # append mode
    f.write("Line 4\n")

with open("data.txt") as f:
    print(f.readline().strip())         # first line
    print(f.readlines())                # remaining lines as list
```

Output

```
Line 1
['Line 2\n', 'Line 3\n', 'Line 4\n']
```

! Careful!

- Mode **"w"** destroys the old content of an existing file. Use **"a"** when you only want to add data.
- After *read()*, the pointer is at the end — a second read() returns an empty string unless you call *seek(0)*.

4. Reading and writing binary files

Binary files store bytes. Open them with **"rb"** / **"wb"**; data read is of type **bytes**, and data to write must also be bytes. A typical use is copying images, audio or any non-text file.

```
with open("photo.jpg", "rb") as src, open("copy.jpg", "wb") as dst:
    data = src.read()           # bytes
    dst.write(data)             # writes the same bytes

b = bytes([72, 105])            # bytes object  -> b'Hi'
print(b, b.decode())
```

Output

```
b'Hi' Hi
```

5. Pickle module — saving whole objects

Pickling (serialisation) converts a Python object (list, dictionary, class object…) into a stream of bytes so it can be stored in a file; **unpickling** converts it back to the object.




```
import pickle
student = {"name": "Asha", "marks": [78, 85, 90]}

with open("student.pkl", "wb") as f:     # binary write
    pickle.dump(student, f)

with open("student.pkl", "rb") as f:     # binary read
    data = pickle.load(f)
print(data)
```

Output

```
{'name': 'Asha', 'marks': [78, 85, 90]}
```

! Careful!

- Only unpickle files from **trusted** sources — a malicious pickle file can run harmful code.

6. Reading and writing CSV files

CSV (Comma Separated Values) stores table-like data: one row per line, values separated by commas. Python's **csv** module handles it.

Function / class	Purpose
`csv.reader(f)`	Reads rows — each row is a **list** of strings.
`csv.writer(f)`	Has writerow(list) and writerows(list of lists) to write data.
`csv.DictReader(f)`	Reads each row as a **dictionary** (header row gives the keys).
`csv.DictWriter(f, fieldnames)`	Writes dictionaries as rows (writeheader() writes the header).

```
import csv
with open("marks.csv", "w", newline="") as f:      # newline="" avoids blank lines
    w = csv.writer(f)
    w.writerow(["Name", "Marks"])
    w.writerows([["Asha", 90], ["Ravi", 75]])

with open("marks.csv") as f:
    for row in csv.reader(f):
        print(row)
```

Output

```
['Name', 'Marks']
['Asha', '90']
['Ravi', '75']
```

7. The os and os.path modules

The **os** module lets a program talk to the operating system — create, delete, rename files and folders. **os.path** works with file paths.

os function	Use	os.path function	Use
`os.getcwd()`	Current working directory	`path.exists(p)`	Does the path exist?
`os.listdir(p)`	List files and folders	`path.isfile(p)`	Is it a file?
`os.mkdir(name)`	Create one folder	`path.isdir(p)`	Is it a folder?
`os.makedirs(path)`	Create nested folders	`path.join(a, b)`	Join path parts safely
`os.rmdir(name)`	Remove empty folder	`path.basename(p)`	File name part of a path
`os.remove(file)`	Delete a file	`path.dirname(p)`	Folder part of a path
`os.rename(old, new)`	Rename file / folder	`path.splitext(p)`	Split name and extension
`os.chdir(path)`	Change directory	`path.getsize(p)`	File size in bytes

```
import os
```

```
print(os.path.exists("data.txt"))
p = os.path.join("reports", "marks.csv")
print(p, os.path.basename(p), os.path.splitext(p))
```

Output

```
False
reports/marks.csv marks.csv ('reports/marks', '.csv')
```

★ Study Tip

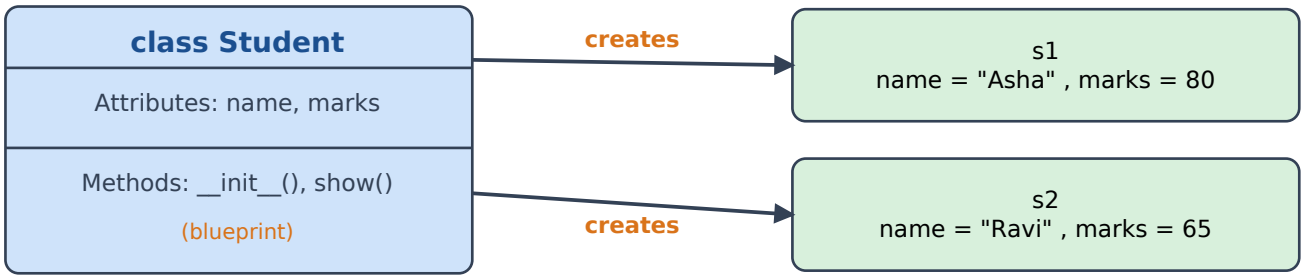
- File handling exam answers score well when you remember the cycle **open → read/write → close** and the table of **modes**.
- Draw the small flow diagram of pickle / file handling — it earns easy marks.

PART B — OBJECT-ORIENTED PROGRAMMING

8. Classes and objects

Object-Oriented Programming (OOP) organises a program around **objects** — things that hold **data** (attributes) and the **functions** that work on that data (methods). Real world: a *Student* has a name and marks (data) and can show details (action).

- **Class** — a blueprint / template (e.g. "Student"). It creates no memory for data by itself.
- **Object** — a real instance built from the class (e.g. Asha, Ravi). Each object has its own copy of the data.
- **Attribute** — variable inside a class/object. **Method** — function inside a class.
- **self** — refers to the **current object**; it must be the first parameter of every instance method.



Creating a class and objects

```
class Student:                      # creating a class
    def show(self):                 # method
        print("I am a student")

s1 = Student()                      # creating an object
s1.show()                           # calling a method with the object
```

Output

```
I am a student
```

9. Constructor method and multiple objects

The **constructor** `__init__(self, ...)` is a special method that runs **automatically** when an object is created. It is used to give initial values to the attributes. (Python has one constructor per class — default values can replace "constructor overloading".)

```
class Student:
    def __init__(self, name, marks):     # constructor
        self.name = name                 # instance attributes
        self.marks = marks

    def show(self):
        print(self.name, "scored", self.marks)

s1 = Student("Asha", 80)     # multiple objects of the same class
s2 = Student("Ravi", 65)
s1.show()
s2.show()
print(s1 is s2)              # two different objects
```

Output

```
Asha scored 80
Ravi scored 65
False
```

10. Class attributes vs instance (data) attributes

	Instance attribute	Class attribute
Defined	Inside `__init__` using `self.x`	Directly inside the class body, outside methods
Belongs to	Each **object** separately	The **class** — shared by all objects
Access	`obj.x`	`ClassName.x` or `obj.x`
Use	Data that differs (name, marks)	Common data (college name, counters)

```
class Student:
    college = "JNTUA"            # class attribute (shared)
    count = 0
    def __init__(self, name):
        self.name = name         # instance attribute (separate)
        Student.count += 1

a = Student("Asha"); b = Student("Ravi")
print(a.college, b.college, Student.count)
Student.college = "JNTU"         # changing it changes it for all objects
print(a.college)
```

Output

```
JNTUA JNTUA 2
JNTU
```

11. Encapsulation

Encapsulation means **wrapping data and the methods that use it into one unit (class)** and **hiding the data** from direct outside access, so it can only be changed in a controlled way. Python uses naming conventions for access levels:

Level	Syntax	Meaning
Public	`self.name`	Accessible from anywhere
Protected	`self._name`	Single underscore — a hint: “for use inside the class and subclasses”
Private	`self.__name`	Double underscore — name is changed internally (*name mangling*), so it cannot be accessed directly from outside

```
class Account:
    def __init__(self, balance):
        self.__balance = balance          # private attribute

    def deposit(self, amt):               # controlled access (setter-like)
        if amt > 0:
            self.__balance += amt

    def get_balance(self):                # getter
        return self.__balance

acc = Account(1000)
```

```
acc.deposit(500)
print(acc.get_balance())
print(acc.__balance)                    # direct access fails
```

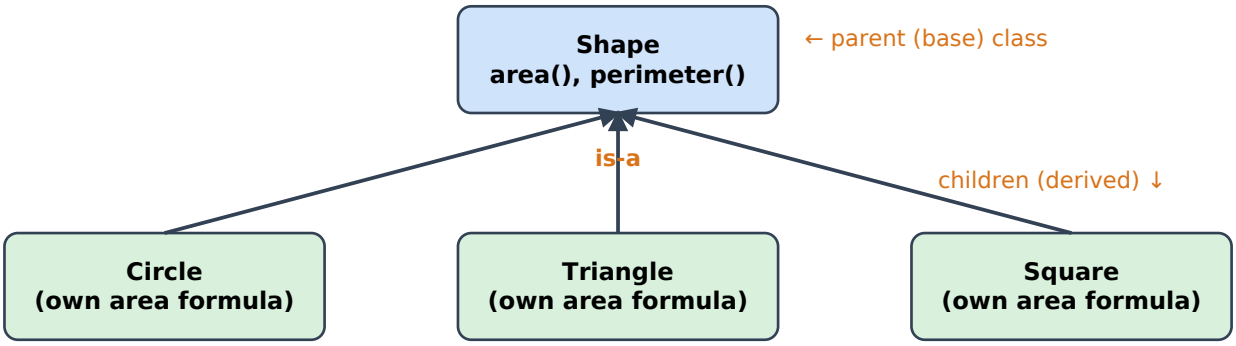
Output

```
1500
AttributeError: 'Account' object has no attribute '__balance'
```

> **i Remember**
>
> - Benefit: data protection, validation (reject negative deposits) and easier maintenance.

12. Inheritance

Inheritance lets a new class (**child / subclass / derived class**) acquire the attributes and methods of an existing class (**parent / superclass / base class**). It gives **code reuse** — write common code once in the parent.



Type	Meaning	Syntax
Single	One parent → one child	`class B(A):`
Multilevel	Grandparent → parent → child	`class C(B): (B inherits A)`
Multiple	Child with two or more parents	`class C(A, B):`
Hierarchical	One parent → many children	`class B(A) class C(A)`

```
class Person:
    def __init__(self, name):
        self.name = name
    def intro(self):
        print("Hi, I am", self.name)

class Student(Person):                  # single inheritance
    def __init__(self, name, roll):
        super().__init__(name)          # call parent constructor
        self.roll = roll
    def show(self):
        print("Roll no:", self.roll)

s = Student("Asha", 101)
s.intro()                               # inherited method
s.show()
print(isinstance(s, Person), issubclass(Student, Person))
```

Output

```
Hi, I am Asha
Roll no: 101
True True
```

13. Polymorphism

Polymorphism = “many forms”. The **same method name or operator behaves differently** for different objects.

- **Method overriding** — a child class redefines a parent’s method with the same name.
- **Duck typing** — if an object has the required method, Python does not care about its class.
- **Operator overloading** — special methods like *__add__* change the meaning of + for objects.

```
class Animal:
    def sound(self): print("Some sound")
class Dog(Animal):
    def sound(self): print("Bow Bow")      # overriding
class Cat(Animal):
    def sound(self): print("Meow")

for a in (Animal(), Dog(), Cat()):
    a.sound()                             # same call, different behaviour

class Point:
    def __init__(self, x): self.x = x
    def __add__(self, o): return Point(self.x + o.x)   # operator overloading
print((Point(3) + Point(4)).x)
```

Output

```
Some sound
Bow Bow
Meow
7
```

✓ Quick Revision

- File cycle: **open → read/write → close**; use **with** for auto-close. Modes: r, w (erase), a (append), r+, rb/wb.
- Reading: read(), readline(), readlines(). Writing: write(), writelines(). seek()/tell() move/show pointer.
- **pickle**: dump() / load() with binary modes. **csv**: reader, writer, DictReader, DictWriter.
- **os**: getcwd, listdir, mkdir, remove, rename · **os.path**: exists, join, isfile, splitext, getsize.
- **Class** = blueprint, **object** = instance, **self** = current object, **__init__** = constructor.
- Instance attribute (per object) vs class attribute (shared). Private data → **__name**.
- **Inheritance** → code reuse (super()). **Polymorphism** → one name, many forms (overriding).

SAMPLE PROGRAMS 18 – 23 (with output)

Program 18

Question

- Write a program to sort words in a file and put them in another file. The output file should have only lower-case words, so any upper-case words from source must be lowered.

Input file: `source.txt`

```
Python is Easy and Python is Powerful
Files are Fun and Useful
```

Program

```
# Read words, convert to lower case, sort and write to another file
with open("source.txt", "r") as f:
    words = f.read().split()            # split on spaces and newlines

words = [w.lower() for w in words]      # every word -> lower case
words.sort()                            # alphabetical order

with open("sorted.txt", "w") as out:
    for w in words:
        out.write(w + "\n")

print("Total words written:", len(words))
```

Output

```
Total words written: 12
```

Content of the output file `sorted.txt`:

```
and
and
are
easy
files
fun
is
is
powerful
python
python
useful
```

Logic: read → split into words → list comprehension with *lower()* → *sort()* → write each word on a new line.

Program 19

Question

- Write a Python program to print each line of a file in reverse order.

Input file: `source.txt`

```
Python is Easy and Python is Powerful
Files are Fun and Useful
```

Program

```
# Method 1: print the lines from last to first
with open("source.txt") as f:
    lines = f.readlines()

print("Lines in reverse order:")
for line in reversed(lines):
    print(line.rstrip("\n"))

# Method 2: reverse the characters of every line
print("\nEach line reversed:")
for line in lines:
    print(line.rstrip("\n")[::-1])
```

Output

```
Lines in reverse order:
Files are Fun and Useful
Python is Easy and Python is Powerful

Each line reversed:
lufrewoP si nohtyP dna ysaE si nohtyP
lufesU dna nuF era seliF
```

i Remember

- If your question means *“reverse the order of lines”* use Method 1; if it means *“reverse each line’s text”* use Method 2. Writing both is safe.

Program 20

Question

- Write a Python program to compute the number of characters, words and lines in a file.

Input file: `source.txt`

```
Python is Easy and Python is Powerful
Files are Fun and Useful
```

Program

```
chars = words = lines = 0
with open("source.txt") as f:
    for line in f:                      # one line at a time
        lines += 1
        words += len(line.split())
        chars += len(line)              # includes the newline character

print("Lines     :", lines)
```

```
print("Words     :", words)
print("Characters:", chars)
```

Output

```
Lines     : 2
Words     : 12
Characters: 63
```

i Remember

- Characters here include the newline (\n) at the end of the line. Use *len(line.rstrip("\n"))* to ignore it.

Program 21

Question

- Write a program to create, display, append, insert and reverse the order of the items in the array.

Program

```
from array import array

arr = array("i", [10, 20, 30, 40])      # create an integer array
print("Created  :", arr.tolist())

print("Display  :", end=" ")
for x in arr:                           # display items
    print(x, end=" ")
print()

arr.append(50)                          # add at the end
print("Appended :", arr.tolist())

arr.insert(2, 25)                       # insert 25 at index 2
print("Inserted :", arr.tolist())

arr.reverse()                           # reverse the order
print("Reversed :", arr.tolist())
```

Output

```
Created  : [10, 20, 30, 40]
Display  : 10 20 30 40
Appended : [10, 20, 30, 40, 50]
Inserted : [10, 20, 25, 30, 40, 50]
Reversed : [50, 40, 30, 25, 20, 10]
```

i Remember

- array("i") stores integers of one type only (type code **i** = int, **f** = float, **d** = double). Other methods: *remove(), pop(), index(), count()*.

Program 22

Question

- Write a program to add, transpose and multiply two matrices.

Program

```
A = [[1, 2, 3],
     [4, 5, 6]]
B = [[7, 8, 9],
     [10, 11, 12]]

# Addition (same order matrices)
add = [[A[i][j] + B[i][j] for j in range(3)] for i in range(2)]
print("A + B =", add)

# Transpose of A (rows become columns)
trans = [[A[i][j] for i in range(2)] for j in range(3)]
print("Transpose of A =", trans)

# Multiplication: A (2x3) x Bt (3x2) -> 2x2
Bt = [[B[i][j] for i in range(2)] for j in range(3)]
mul = [[0, 0], [0, 0]]
for i in range(2):
    for j in range(2):
        for k in range(3):
            mul[i][j] += A[i][k] * Bt[k][j]
print("A x B^T =", mul)
```

Output

```
A + B = [[8, 10, 12], [14, 16, 18]]
Transpose of A = [[1, 4], [2, 5], [3, 6]]
A x B^T = [[50, 68], [122, 167]]
```

✓ Quick Revision

- Addition needs **same order**. Multiplication needs **columns of first = rows of second**; result order = rows of first × columns of second.

Program 23

Question

- Write a Python program to create a class that represents a shape. Include methods to calculate its area and perimeter. Implement subclasses for different shapes like circle, triangle, and square.

Program

```
import math

class Shape:                                    # parent class
    def area(self):
        return 0
    def perimeter(self):
        return 0

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
```

```
        return math.pi * self.r ** 2
    def perimeter(self):
        return 2 * math.pi * self.r

class Triangle(Shape):
    def __init__(self, a, b, c):
        self.a, self.b, self.c = a, b, c
    def area(self):                          # Heron's formula
        s = (self.a + self.b + self.c) / 2
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))
    def perimeter(self):
        return self.a + self.b + self.c

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2
    def perimeter(self):
        return 4 * self.side

for s in (Circle(5), Triangle(3, 4, 5), Square(4)):
    print(f"{type(s).__name__:<9} Area = {s.area():7.2f}   Perimeter = {s.perimeter():6.2f}")
```

Output

```
Circle    Area =   78.54   Perimeter =  31.42
Triangle  Area =    6.00   Perimeter =  12.00
Square    Area =   16.00   Perimeter =  16.00
```

i Remember

- Concepts used: **inheritance** (subclasses of Shape), **method overriding** and **polymorphism** (same *area()* call works for every shape).

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. Differentiate text and binary files. 2. List any four file opening modes. 3. What is the use of the *with* statement? 4. What is pickling? 5. What is a constructor? 6. Differentiate class and instance attributes. 7. What is encapsulation? 8. What is method overriding?	1. Explain the methods to read and write text files with examples. 2. Explain the *pickle* and *csv* modules with programs. 3. Explain *os* and *os.path* functions with examples. 4. Explain classes, objects and constructors with a program. 5. Explain the types of inheritance with examples. 6. Explain encapsulation and polymorphism with programs. 7. Write programs 18–23 from the sample experiments.

UNIT V

Introduction to Data Science

Functional programming · JSON and XML in Python · NumPy · Pandas · Matplotlib basics · Sample Programs 24–30

➤ What you will learn in this unit

- Functional-style Python: **lambda, map, filter, reduce** and comprehensions.
- Exchanging data using **JSON** and **XML** with the json and ElementTree modules.
- Fast numerical work with **NumPy arrays**, and table-style analysis with **Pandas** DataFrames.
- Plotting data with **Matplotlib** and solutions of sample programs **24 to 30** with real output.

1. What is Data Science?

Data Science is the process of collecting data, cleaning it, analysing it and turning it into useful decisions. Python is the most popular language for it because of its simple syntax and powerful libraries.



Python tools: json / csv → Pandas → NumPy → Matplotlib

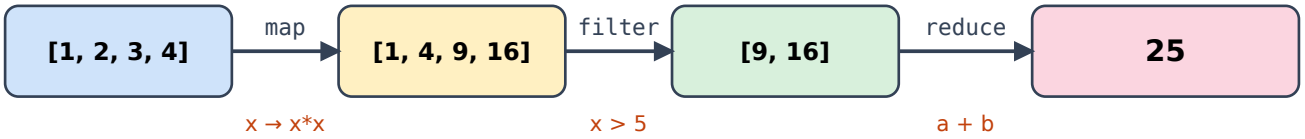
Library	Purpose
NumPy	Fast multi-dimensional arrays and mathematics
Pandas	Tables (DataFrames), data cleaning, selection and analysis
Matplotlib	Graphs and charts
json / xml	Reading and writing JSON and XML data formats

2. Functional Programming in Python

Functional programming is a style where a program is built by combining **functions**. Functions are treated as values: they can be stored in variables, passed to other functions and returned from functions.

- **Pure function** — same input always gives same output and changes nothing outside (no side effects).
- **First-class functions** — functions can be assigned to variables and passed as arguments.
- **Higher-order function** — a function that takes another function as argument (map, filter, sorted with key).
- **Lambda** — a small **anonymous** one-line function: `lambda args: expression`

Tool	What it does	Example
`map(f, seq)`	Applies f to **every** item	`map(lambda x: x*x, [1,2,3])` → 1,4,9
`filter(f, seq)`	Keeps items for which f is **True**	`filter(lambda x: x>2, [1,2,3])` → 3
`reduce(f, seq)`	Combines all items into **one value** (functools)	`reduce(lambda a,b: a+b, [1,2,3])` → 6
`[x for x in s if c]`	List comprehension — short form of map + filter	`[x*x for x in s if x%2]`
`zip(a, b)`	Pairs items of two sequences	`zip([1,2],["a","b"])`




```
from functools import reduce
nums = [1, 2, 3, 4, 5]

square = lambda x: x * x                        # lambda function
print(list(map(square, nums)))                  # map
print(list(filter(lambda x: x % 2 == 0, nums)))# filter
print(reduce(lambda a, b: a + b, nums))         # reduce
print([x * x for x in nums if x % 2 == 1])      # comprehension
print(sorted(["pear", "fig", "apple"], key=len))   # function as argument

def squares(n):                                 # generator: yields values lazily
    for i in range(n):
        yield i * i
print(list(squares(5)))
```

Output

```
[1, 4, 9, 16, 25]
[2, 4]
15
[1, 9, 25]
['fig', 'pear', 'apple']
[0, 1, 4, 9, 16]
```

i Remember

- In Python 3, *map()* and *filter()* return iterators — wrap them in *list()* to see the values.

3. JSON in Python

JSON (JavaScript Object Notation) is a lightweight text format for exchanging data (web APIs, config files). It looks like a Python dictionary. Python's **json** module converts between JSON text and Python objects.

Function	Purpose
`json.loads(s)`	JSON **string** → Python object (dict / list)
`json.dumps(obj)`	Python object → JSON **string** (use *indent=2* for neat output)
`json.load(f)`	Read JSON from a **file** → Python object
`json.dump(obj, f)`	Write a Python object to a **file** as JSON

JSON	object { }	array [ ]	string "..."	number	true / false	null
Python	dict	list	str	int / float	True / False	None

```
import json
text = '{"name": "Asha", "marks": [90, 85], "passed": true, "address": null}'
data = json.loads(text)                  # JSON string -> Python dict
print(data["name"], data["marks"][0], data["passed"], data["address"])
print(json.dumps(data, indent=2))        # Python -> JSON string
```

Output

```
Asha 90 True None
{
  "name": "Asha",
  "marks": [
    90,
    85
  ],
  "passed": true,
  "address": null
}
```

4. XML in Python

XML (eXtensible Markup Language) stores data in a tree of **tags** (elements) with attributes and text. Python reads it with **xml.etree.ElementTree**.

ElementTree item	Meaning
`ET.parse("f.xml")`	Reads an XML file; use *.getroot()* to get the root element
`ET.fromstring(s)`	Parses an XML string and returns the root element
`elem.tag / elem.text`	Tag name and the text inside the element
`elem.get("id") / elem.attrib`	Attribute value / all attributes as a dictionary
`elem.find("t") / findall("t")`	First matching child / list of all matching children
`ET.SubElement(parent, "t")`	Creates a new child element (to build XML)

```
import xml.etree.ElementTree as ET
xml_data = """<students>
  <student id="1"><name>Asha</name><marks>90</marks></student>
  <student id="2"><name>Ravi</name><marks>75</marks></student>
</students>"""
root = ET.fromstring(xml_data)
print(root.tag)
for s in root.findall("student"):
    print(s.get("id"), s.find("name").text, s.find("marks").text)
```

Output

```
students
1 Asha 90
2 Ravi 75
```

	JSON	XML
Syntax	Key-value pairs, braces and brackets	Opening and closing tags
Size / speed	Shorter and faster to parse	Longer (more verbose)
Data types	Numbers, strings, booleans, arrays	Everything is text
Common use	Web APIs, configuration	Documents, older systems, RSS, SOAP

5. NumPy with Python

NumPy (Numerical Python) provides the **ndarray** — a fast, fixed-type, multi-dimensional array. It is much faster than Python lists and allows **vectorised** operations (apply an operation to the whole array without loops).

	Python list	NumPy array
Data types	Can mix types	All elements have **one** dtype
Speed / memory	Slower, more memory	Much faster, compact
*[1,2]*2 / a*2*	Repeats the list → [1,2,1,2]	Multiplies each element → [2,4]

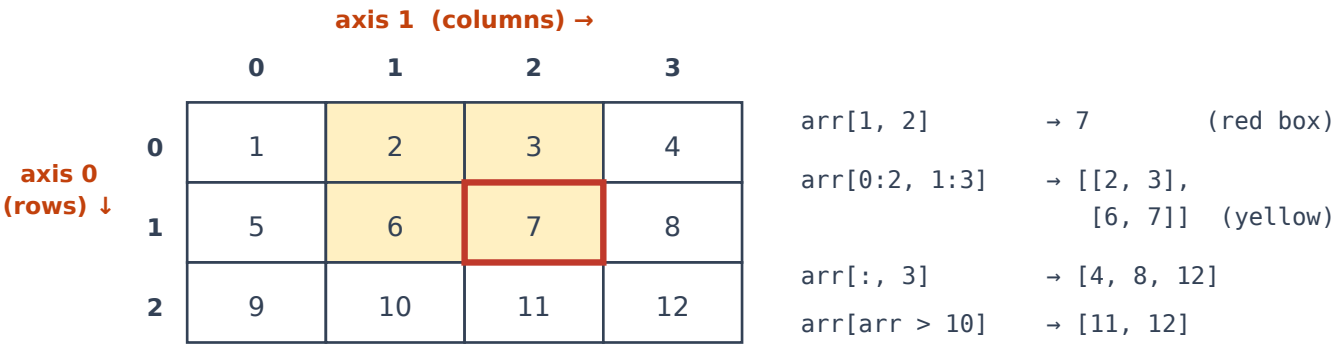
Creating arrays

Function	Result
`np.array([1,2,3])`	Array from a list / tuple (the main way)
`np.zeros((2,3)) np.ones((2,3))`	Arrays filled with 0 / 1
`np.arange(0, 10, 2)`	Like range(): 0,2,4,6,8
`np.linspace(0, 1, 5)`	5 equally spaced numbers from 0 to 1
`np.eye(3)`	Identity matrix
`np.random.rand(2,2)`	Random numbers between 0 and 1

Important attributes

Attribute	Meaning
`a.ndim`	Number of dimensions (axes)
`a.shape`	Tuple of size in each dimension, e.g. (3, 4) = 3 rows, 4 columns
`a.size`	Total number of elements
`a.dtype`	Data type of the elements (int64, float64 …)

Indexing and slicing




```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(arr.shape, arr.ndim, arr.size, arr.dtype)
print(arr[1, 2])                  # row 1, column 2
print(arr[0:2, 1:3])              # slicing: rows 0-1, columns 1-2
print(arr[:, 3])                  # whole last column
print(arr[arr > 10])              # Boolean indexing
print(arr * 2)                    # vectorised operation
print(arr.reshape(4, 3).shape)    # change shape
```

Output

```
(3, 4) 2 12 int64
7
[[2 3]
 [6 7]]
[ 4  8 12]
[11 12]
[[ 2  4  6  8]
 [10 12 14 16]
 [18 20 22 24]]
(4, 3)
```

Aggregate functions

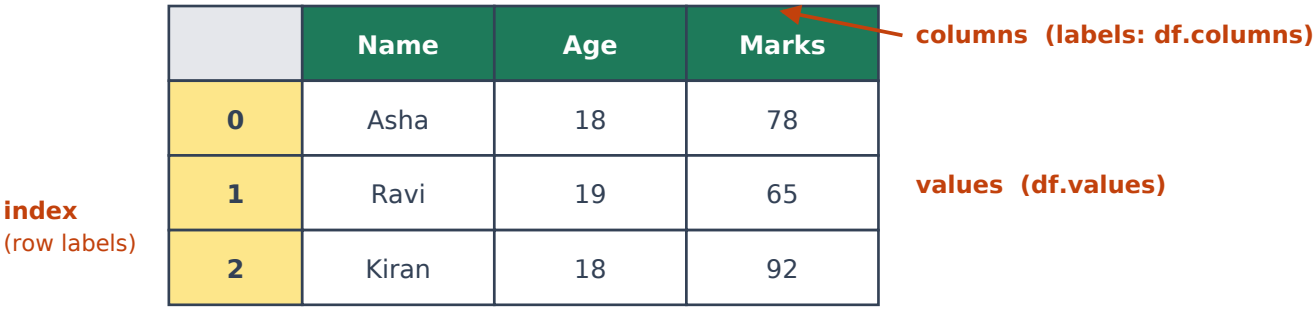
Function	Result	Function	Result
`a.min()`	Smallest value	`a.sum()`	Total of all elements
`a.max()`	Largest value	`a.cumsum()`	Running (cumulative) sum
`a.mean()`	Average	`a.sum(axis=0)`	Column-wise total (axis=1 → row-wise)

★ Study Tip

- Remember **axis 0 = rows direction (down)**, **axis 1 = columns direction (across)**.
- Slicing *a[start:stop]* gives a **view** (no copy). Use *.copy()* if you need an independent array.

6. Pandas

Pandas is built on NumPy and gives two main structures: **Series** (one-dimensional labelled array) and **DataFrame** (two-dimensional table with rows and columns, like an Excel sheet).



	Name	Age	Marks
0	Asha	18	78
1	Ravi	19	65
2	Kiran	18	92

DataFrame: every column is a Series with its own data type

Creating and exploring

```
import pandas as pd
s = pd.Series([10, 20, 30], index=["a", "b", "c"])     # Series
print(s["b"])

df = pd.DataFrame({"Name": ["Asha", "Ravi", "Kiran"],
                   "Age": [18, 19, 18],
                   "Marks": [78, 65, 92]})              # from a dictionary
print(df)
print(df.shape, list(df.columns))
print(df.describe())                                    # quick statistics
```

Output

```
20
    Name  Age  Marks
0   Asha   18     78
1   Ravi   19     65
2  Kiran   18     92
(3, 3) ['Name', 'Age', 'Marks']
             Age      Marks
count   3.000000   3.000000
mean   18.333333  78.333333
std     0.577350  13.503086
min    18.000000  65.000000
25%    18.000000  71.500000
50%    18.000000  78.000000
75%    18.500000  85.000000
max    19.000000  92.000000
```

Selecting data

Operation	Code	Gives
One column	`df["Marks"]`	A Series
Many columns	`df[["Name","Marks"]]`	A DataFrame
Row by label	`df.loc[1]`	Row whose index label is 1
By position	`df.iloc[0:2, 0:2]`	First 2 rows and first 2 columns
By condition	`df[df["Marks"] > 70]`	Rows where Marks > 70
First / last rows	`df.head(n) df.tail(n)`	First / last n rows (default 5)

```
import pandas as pd
df = pd.DataFrame({"Name": ["Asha", "Ravi", "Kiran"], "Age": [18, 19, 18], "Marks": [78, 65, 92]})
print(df[df["Marks"] > 70])                 # filter rows
df["Result"] = df["Marks"] >= 70            # add a new column
print(df.sort_values("Marks", ascending=False).head(2))
df.to_csv("marks.csv", index=False)         # save to CSV
```

```
print(pd.read_csv("marks.csv").shape)       # read it back (rows, columns)
```

Output

```
    Name  Age  Marks
0   Asha   18     78
2  Kiran   18     92
    Name  Age  Marks  Result
2  Kiran   18     92    True
0   Asha   18     78    True
(3, 4)
```

i Remember

- **loc** uses **labels**, **iloc** uses **integer positions**. Missing data: *df.isnull()*, *df.dropna()*, *df.fillna(value)*.

7. Plotting with Matplotlib

Matplotlib draws graphs. The common functions are in **matplotlib.pyplot** (imported as *plt*).

Function	Purpose
`plt.plot(x, y)`	Line graph — shows how y changes with x
`plt.scatter(x, y)`	Scatter plot — one dot per (x, y) pair, shows relationship / correlation
`plt.bar(x, h) plt.hist(d)`	Bar chart / histogram
`plt.xlabel() ylabel() title()`	Axis labels and title
`plt.show()`	Displays the figure

✓ Quick Revision

- **Functional**: lambda (anonymous function), map (apply), filter (select), reduce (combine, from functools), comprehension.
- **JSON**: loads/dumps (strings), load/dump (files). object→dict, array→list, true→True, null→None.
- **XML**: ElementTree — fromstring/parse, find/findall, .text, .get(attr).
- **NumPy**: np.array(); ndim, shape, size, dtype; slicing a[r1:r2, c1:c2]; Boolean index a[a>5]; min/max/sum/cumsum.
- **Pandas**: Series (1-D), DataFrame (2-D); head(); loc (label) / iloc (position); df[condition]; read_csv / to_csv.
- **Matplotlib**: plot (line), scatter (dots), labels, show().

SAMPLE PROGRAMS 24 – 30 (with output)

Program 24

Question

- Python program to check whether a JSON string contains complex object or not.

Program

```
import json

def has_complex(data):
    # complex object = a value that is itself a dictionary (object) or list (array)
    if isinstance(data, dict):
        values = data.values()
    elif isinstance(data, list):
        values = data
    else:
        return False
    return any(isinstance(v, (dict, list)) for v in values)

s1 = '{"name": "Ravi", "age": 20}'
s2 = '{"name": "Ravi", "marks": {"maths": 90, "physics": 85}}'
s3 = '{"name": "Ravi", "subjects": ["Maths", "Physics"]}'

for s in (s1, s2, s3):
    data = json.loads(s)                    # JSON string -> Python object
    result = "Complex object" if has_complex(data) else "Simple (no complex object)"
    print(s, "->", result)
```

Output

```
{"name": "Ravi", "age": 20} -> Simple (no complex object)
{"name": "Ravi", "marks": {"maths": 90, "physics": 85}} -> Complex object
{"name": "Ravi", "subjects": ["Maths", "Physics"]} -> Complex object
```

i Remember

- A JSON value is **simple** if it is a string, number, boolean or null. If it is an **object { }** or **array [ ]** then it is a **complex** (nested) value.

Program 25

Question

- Python program to demonstrate NumPy arrays creation using array() function.

Program

```
import numpy as np

a = np.array([10, 20, 30, 40])                      # 1-D array from a list
b = np.array([[1, 2, 3], [4, 5, 6]])                # 2-D array (list of lists)
c = np.array((7, 8, 9))                             # from a tuple
d = np.array([1, 2, 3], dtype=float)                # with a given data type
e = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  # 3-D array

print("1-D array :", a)
print("2-D array :\n", b)
print("From tuple:", c)
print("Float type:", d)
```

```
print("3-D array :\n", e)
```

Output

```
1-D array : [10 20 30 40]
2-D array :
 [[1 2 3]
 [4 5 6]]
From tuple: [7 8 9]
Float type: [1. 2. 3.]
3-D array :
 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Program 26

Question

- Python program to demonstrate use of ndim, shape, size, dtype.

Program

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr)
print("ndim  (dimensions)  :", arr.ndim)
print("shape (rows, cols) :", arr.shape)
print("size  (elements)   :", arr.size)
print("dtype (data type)  :", arr.dtype)

f = np.array([1.5, 2.5, 3.5])
print("Float array dtype  :", f.dtype)
g = f.astype(int)                       # change the data type
print("After astype(int)  :", g, g.dtype)
```

Output

```
Array:
 [[1 2 3]
 [4 5 6]]
ndim  (dimensions)  : 2
shape (rows, cols) : (2, 3)
size  (elements)   : 6
dtype (data type)  : int64
Float array dtype  : float64
After astype(int)  : [1 2 3] int64
```

i Remember

- The integer dtype may print as **int32** on some Windows systems and **int64** on others — both are correct.

Program 27

Question

- Python program to demonstrate basic slicing, integer and Boolean indexing.

Program

```
import numpy as np

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

print("Basic slicing")
print("arr[0:2, 1:3] =\n", arr[0:2, 1:3])    # rows 0-1, columns 1-2
print("arr[:, 2]     =", arr[:, 2])           # all rows, column 2
print("arr[1]        =", arr[1])              # row 1

print("\nInteger indexing")
print("arr[1, 2]              =", arr[1, 2])
print("arr[[0, 1, 2], [1, 2, 3]] =", arr[[0, 1, 2], [1, 2, 3]])  # picks (0,1),(1,2),(2,3)

print("\nBoolean indexing")
print("arr > 6        =\n", arr > 6)
print("arr[arr > 6]   =", arr[arr > 6])
print("arr[arr % 2 == 0] =", arr[arr % 2 == 0])
```

Output

```
Basic slicing
arr[0:2, 1:3] =
 [[2 3]
 [6 7]]
arr[:, 2]     = [ 3  7 11]
arr[1]        = [5 6 7 8]

Integer indexing
arr[1, 2]              = 7
arr[[0, 1, 2], [1, 2, 3]] = [ 2  7 12]

Boolean indexing
arr > 6        =
 [[False False False False]
 [False False  True  True]
 [ True  True  True  True]]
arr[arr > 6]   = [ 7  8  9 10 11 12]
arr[arr % 2 == 0] = [ 2  4  6  8 10 12]
```

Program 28

Question

- Python program to find min, max, sum, cumulative sum of array.

Program

```
import numpy as np

a = np.array([4, 7, 1, 9, 3])
print("Array     :", a)
print("Minimum   :", a.min(), " at index", a.argmin())
print("Maximum   :", a.max(), " at index", a.argmax())
print("Sum       :", a.sum())
print("Cumulative sum:", a.cumsum())
```

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print("\n2-D array:\n", b)
print("Column-wise sum (axis=0):", b.sum(axis=0))
print("Row-wise sum    (axis=1):", b.sum(axis=1))
print("Cumulative sum of all   :", b.cumsum())
```

Output

```
Array     : [4 7 1 9 3]
Minimum   : 1  at index 2
Maximum   : 9  at index 3
Sum       : 24
Cumulative sum: [ 4 11 12 21 24]

2-D array:
 [[1 2 3]
 [4 5 6]]
Column-wise sum (axis=0): [5 7 9]
Row-wise sum    (axis=1): [ 6 15]
Cumulative sum of all   : [ 1  3  6 10 15 21]
```

Program 29

Question

- Create a dictionary with at least five keys and each key represent value as a list where this list contains at least ten values and convert this dictionary as a pandas data frame and explore the data through the data frame as follows: a) Apply head() function to the pandas data frame b) Perform various data selection operations on Data Frame.

Program

```
import pandas as pd

data = {
    "Roll": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Name": ["Asha","Ravi","Kiran","Divya","Eswar","Farah","Gopi","Hema","Imran","Jaya"],
    "Age": [18, 19, 18, 20, 19, 18, 20, 19, 18, 19],
    "Marks": [78, 65, 92, 55, 88, 70, 45, 81, 60, 95],
    "Attendance": [85, 72, 95, 60, 90, 78, 50, 88, 68, 97],
}

df = pd.DataFrame(data)                  # dictionary -> DataFrame
print("Full DataFrame:\n", df)

print("\n(a) head() - first 5 rows:\n", df.head())
print("\nhead(3):\n", df.head(3))

print("\n(b) Data selection operations")
print("\n1. Single column  df['Name']:")
print(df["Name"].head(3))
print("\n2. Multiple columns  df[['Name', 'Marks']]:")
print(df[["Name", "Marks"]].head(3))
print("\n3. Row by label  df.loc[2]:")
print(df.loc[2])
print("\n4. By position  df.iloc[0:3, 1:4]:")
print(df.iloc[0:3, 1:4])
print("\n5. Condition  df[df['Marks'] > 80]:")
print(df[df["Marks"] > 80])
print("\n6. Rows and columns together  df.loc[df['Age'] == 18, ['Name', 'Marks']]:")
print(df.loc[df["Age"] == 18, ["Name", "Marks"]])
```

Output

```
Full DataFrame:
    Roll   Name  Age  Marks  Attendance
0     1   Asha   18     78          85
1     2   Ravi   19     65          72
2     3  Kiran   18     92          95
3     4  Divya   20     55          60
4     5  Eswar   19     88          90
5     6  Farah   18     70          78
6     7   Gopi   20     45          50
7     8   Hema   19     81          88
8     9  Imran   18     60          68
9    10   Jaya   19     95          97

(a) head() - first 5 rows:
    Roll   Name  Age  Marks  Attendance
0     1   Asha   18     78          85
1     2   Ravi   19     65          72
2     3  Kiran   18     92          95
3     4  Divya   20     55          60
4     5  Eswar   19     88          90

head(3):
    Roll   Name  Age  Marks  Attendance
0     1   Asha   18     78          85
1     2   Ravi   19     65          72
2     3  Kiran   18     92          95

(b) Data selection operations

1. Single column  df['Name']:
0     Asha
1     Ravi
2    Kiran
Name: Name, dtype: str

2. Multiple columns  df[['Name', 'Marks']]:
    Name  Marks
0   Asha     78
1   Ravi     65
2  Kiran     92

3. Row by label  df.loc[2]:
Roll              3
Name          Kiran
Age              18
Marks            92
Attendance       95
Name: 2, dtype: object

4. By position  df.iloc[0:3, 1:4]:
    Name  Age  Marks
0   Asha   18     78
1   Ravi   19     65
2  Kiran   18     92

5. Condition  df[df['Marks'] > 80]:
   Roll   Name  Age  Marks  Attendance
2     3  Kiran   18     92          95
4     5  Eswar   19     88          90
7     8   Hema   19     81          88
9    10   Jaya   19     95          97

6. Rows and columns together  df.loc[df['Age'] == 18, ['Name', 'Marks']]:
    Name  Marks
0   Asha     78
2  Kiran     92
5  Farah     70
8  Imran     60
```

Program 30

Question

- Select any two columns from the above data frame, and observe the change in one attribute with respect to other attribute with scatter and plot operations in matplotlib.

Program

```
import pandas as pd

data = {
    "Roll": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Name": ["Asha","Ravi","Kiran","Divya","Eswar","Farah","Gopi","Hema","Imran","Jaya"],
    "Age": [18, 19, 18, 20, 19, 18, 20, 19, 18, 19],
    "Marks": [78, 65, 92, 55, 88, 70, 45, 81, 60, 95],
    "Attendance": [85, 72, 95, 60, 90, 78, 50, 88, 68, 97],
}

import matplotlib.pyplot as plt

df = pd.DataFrame(data).sort_values("Attendance")   # sort so the line graph is smooth
x = df["Attendance"]                                # attribute 1
y = df["Marks"]                                     # attribute 2

fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].scatter(x, y, color="crimson")                # scatter operation
ax[0].set_title("Scatter plot")
ax[0].set_xlabel("Attendance (%)")
ax[0].set_ylabel("Marks")

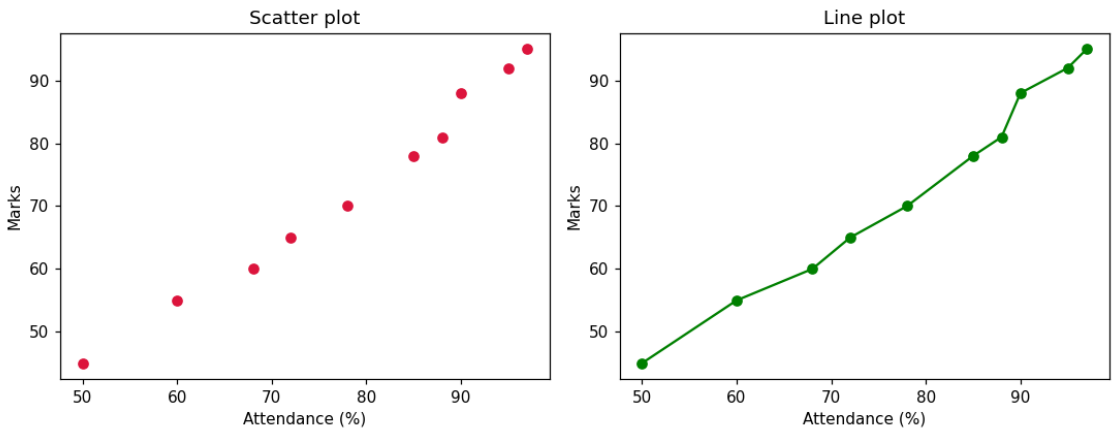
ax[1].plot(x, y, marker="o", color="green")         # plot (line) operation
ax[1].set_title("Line plot")
ax[1].set_xlabel("Attendance (%)")
ax[1].set_ylabel("Marks")

plt.tight_layout()
plt.show()
print("Correlation between Attendance and Marks:", round(x.corr(y), 3))
```

Output

```
Correlation between Attendance and Marks: 0.992
```

Graph produced



i Remember

- **Observation:** the scatter points and the line both rise from left to right — as **Attendance increases, Marks also increase** (strong positive correlation).

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. What is a lambda function? 2. Differentiate map() and filter(). 3. What is JSON? Name any two json functions. 4. Differentiate JSON and XML. 5. What is a NumPy array? List any four attributes. 6. What is a DataFrame? 7. Differentiate loc and iloc. 8. What is the use of head()?	1. Explain functional programming with map, filter, reduce and lambda. 2. Explain JSON and XML handling in Python with examples. 3. Explain NumPy array creation, indexing and slicing with programs. 4. Explain Pandas Series and DataFrame with data selection operations. 5. Explain scatter and line plots using Matplotlib. 6. Write programs 24–30 from the sample experiments.